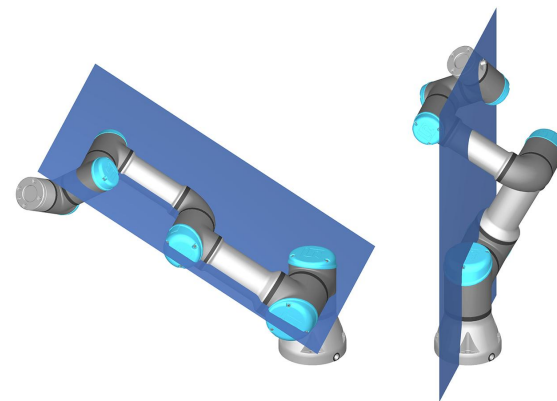
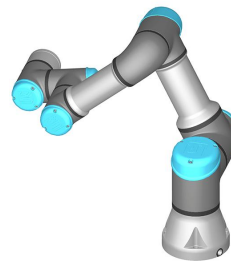
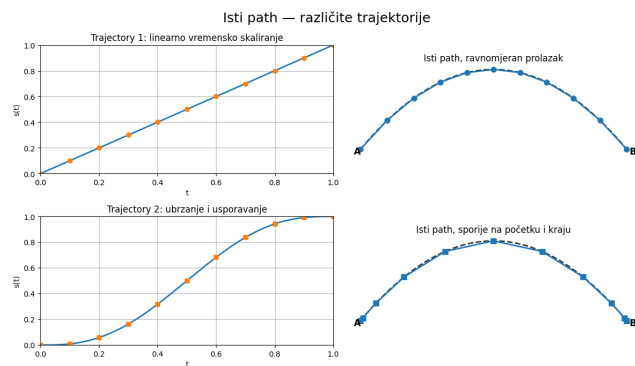
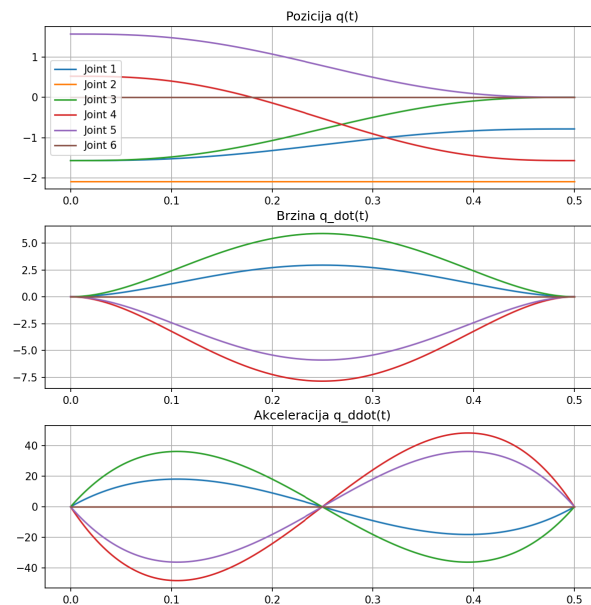
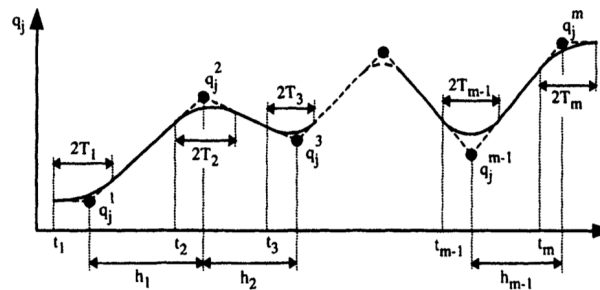
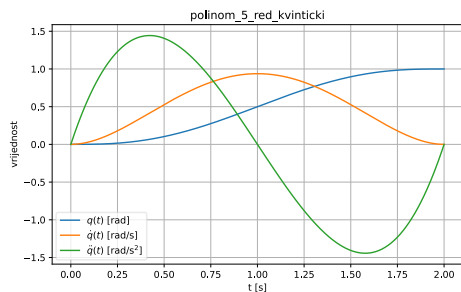


Generiranje trajektorija

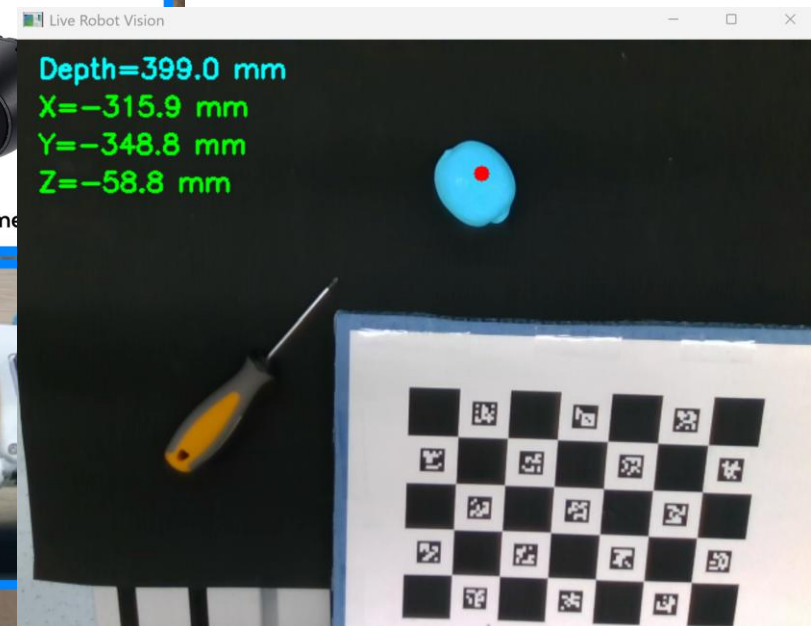


Tara Knežević,
mag. ing. mech.



Humanoidni roboti i biomimetički sustavi, Zagreb, 2025./2026.

Odgovorili smo na pitanje: Kako dobiti 3D podatke o sceni iz 2D slike?

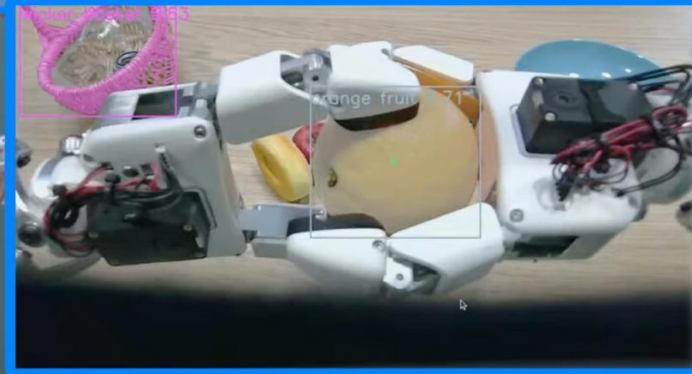


Postavljamo novo pitanje: Koji je zakon gibanja (pozicija, brzina, akceleracija) između zadanih stanja?

$q_{ref} = ?$



Gemini 336 Stereo Vision Camera



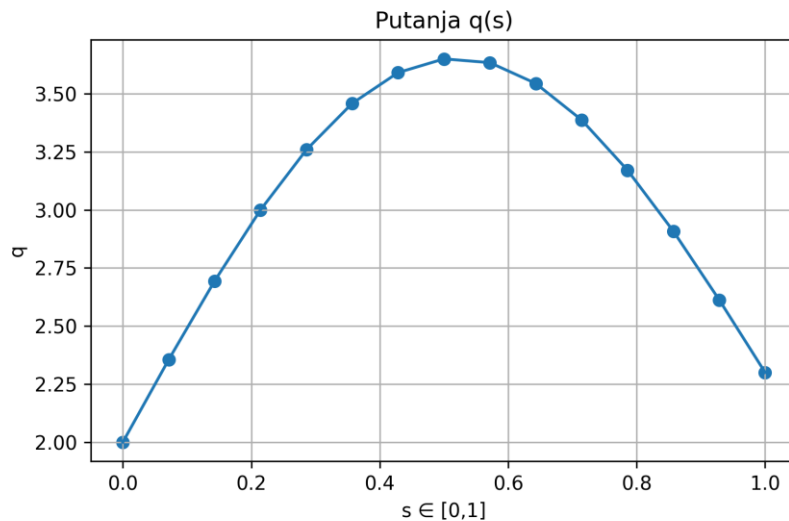
Osnovni pojmovi

PONAVLJANJE + FRAMING PROBLEMA

Putanja vs trajektorija

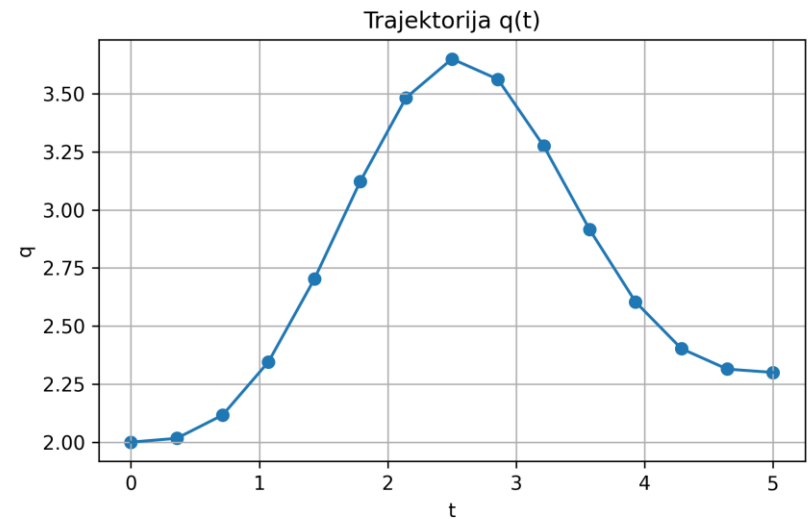
PUTANJA $q(s)$:

- Geometrija kretanja
(redoslijed točaka u prostoru)
- Ne ovisi o vremenu: $s \in [0, 1]$



TRAJEKTORIJA $q(t)$:

- Vremenski skalirana putanja
- Ovisi o vremenu (definira
kada i kako brzo se dolazi u
neku točku): $q(t) = q(s(t))$



Vremensko skaliranje $s(t)$ oblikuje dinamiku gibanja — ista putanja, drugačija trajektorija

Vremensko skaliranje i chain rule

- Funkcija $s(t)$ dodjeljuje vrijednost s za svaki trenutak t
- Različiti oblici $s(t) \rightarrow$ različita dinamika gibanja



Što sve spada pod dinamiku gibanja?

Brzina ($\dot{q}(t)$), **akceleracija** ($\ddot{q}(t)$), a implicitno sile i momenti u sustavu

$q(t) = q(s(t)) \longrightarrow$ Veza putanje i trajektorije

- CHAIN RULE je pravilo za deriviranje složenih funkcija
 - Ovim pravilom derivirat ćemo vezu putanje i trajektorije kako bi dobili **brzinu** i **akceleraciju duž trajektorije** koristeći putanju i vremensko skaliranje

$$\frac{dq}{dt} = \frac{dq}{ds} \frac{ds}{dt} \rightarrow \dot{q} = \frac{dq}{ds} \cdot \dot{s} \longrightarrow \text{Brzina duž trajektorije}$$

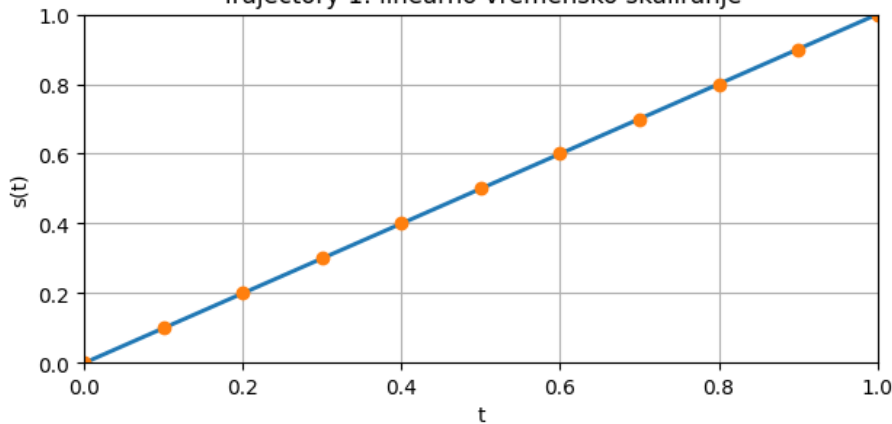
$$\frac{d^2q}{dt^2} = \frac{dq}{ds} \frac{d^2s}{dt^2} + \frac{d^2q}{ds^2} \left(\frac{ds}{dt} \right)^2 \rightarrow \ddot{q} = \frac{dq}{ds} \cdot \ddot{s} + \frac{d^2q}{ds^2} \dot{s}^2 \longrightarrow \text{Akceleracija duž trajektorije}$$

Vremensko skaliranje i chain rule

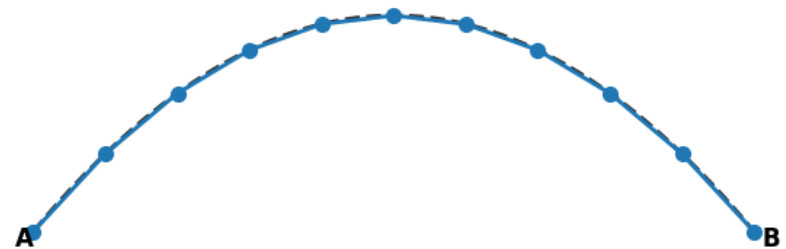
primjer

Isti path — različite trajektorije

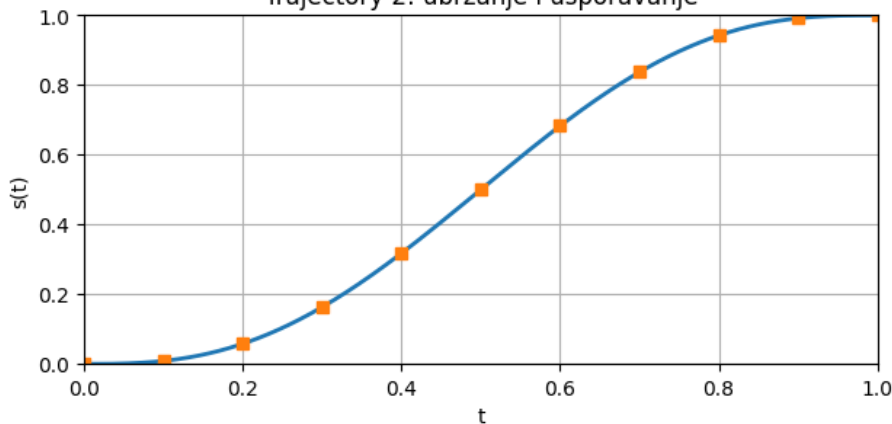
Trajectory 1: linearno vremensko skaliranje



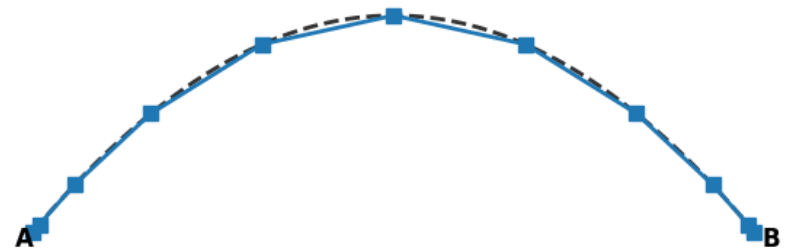
Isti path, ravnomjieran prolazak



Trajectory 2: ubrzanje i usporavanje



Isti path, sporije na početku i kraju



Joint space vs Task space

JOINT SPACE:

- Opis robota kutovima zglobova:

$$\mathbf{q} = [q_1, q_2, \dots, q_n]$$

- + Ograničenja zglobova su trivijalna:

$$q_{i,min} \leq q_i \leq q_{i,max}$$

TASK SPACE:

- Opis robota pozom TCP-a:

$$\mathbf{X} = [x, y, z, R_x, R_y, R_z]$$

- + Intuitivno planiranje u fizičkom prostoru

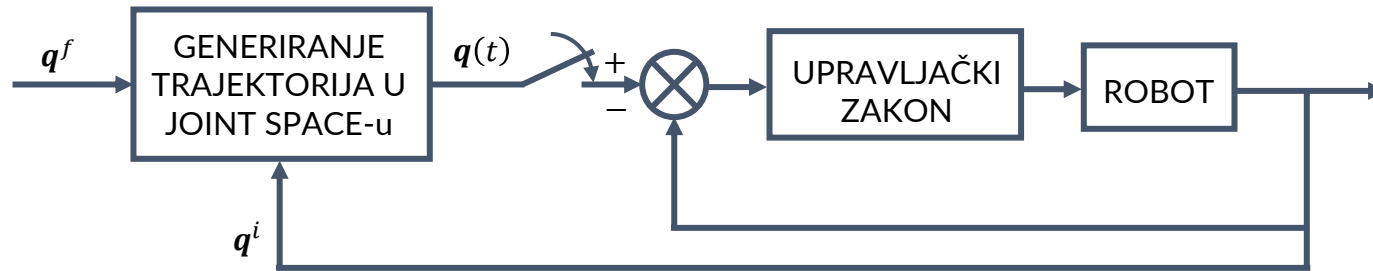
Zašto ipak uvijek preferiramo joint space?

Zbog upravljačkih zakona!

Joint space vs Task space

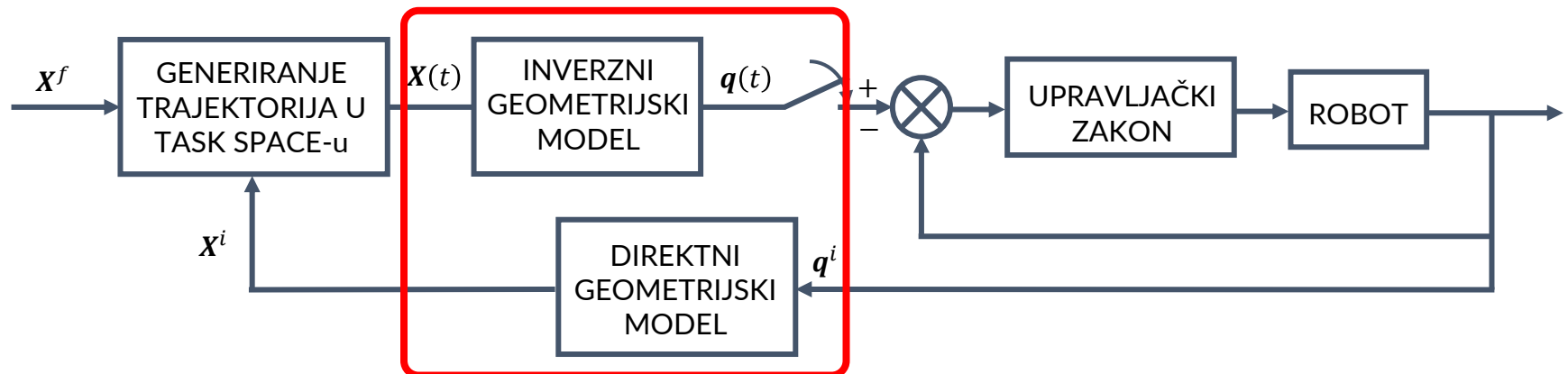
- Cilj generiranja trajektorija je određivanje reference za određeni upravljački zakon

JOINT SPACE:



Superskript „f” označava konačno, a „i” inicijalno stanje

TASK SPACE:



Singulariteti – zašto izbor prostora nije trivijalan

Joint space vs. Task space

- Singularitet = konfiguracija u kojoj robot gubi jedan ili više stupnjeva slobode
- **Matematički:** Jacobian gubi rang ($\det(J) = 0 \rightarrow J^{-1}$ ne postoji)

Što nam Jacobian govori?

$\dot{x} = J(q)\dot{q} \longrightarrow$ Veza brzina zglobova i brzine TCP-a

- Posljedica singulariteta: $\dot{x} = J(q)\dot{q} \rightarrow \dot{q} = J^{-1}\dot{x}$

$$J^{-1} = \frac{1}{\det(J)} \text{adj}(J) \xrightarrow{\text{Dijelimo s } \det(J) \approx 0} J^{-1} \rightarrow \infty$$

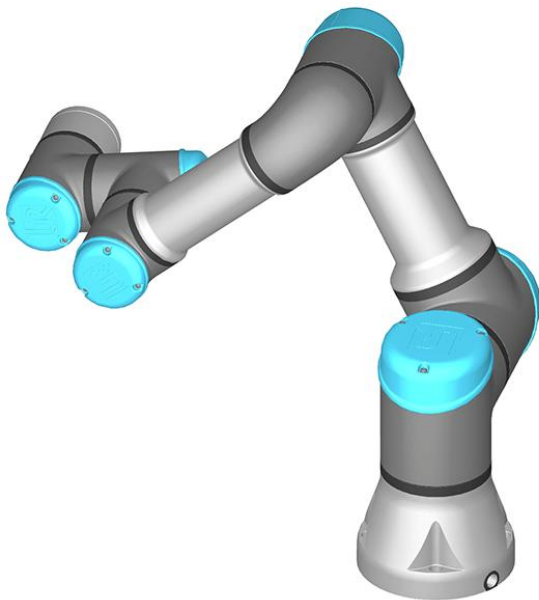
Male promjene $\dot{x}$ množene s ogromnim J^{-1} daju ogromne $\dot{q}$

- **Fizikalno:** blizu singulariteta, TCP "gubi polugu" (jedan smjer kretanja zahtijeva sve veće i veće gibanje zglobova za sve manji i manji efekt u prostoru). Robot se mora "vrtjeti" puno da bi se malo pomaknuo u kritičnom smjeru.
- **Praktična posljedica u planiranju:** zadavanje putanje u task space-u koja prolazi kroz singularitet $\rightarrow$ numerički algoritam za inverznu kinematiku generira nerealne brzine zglobova

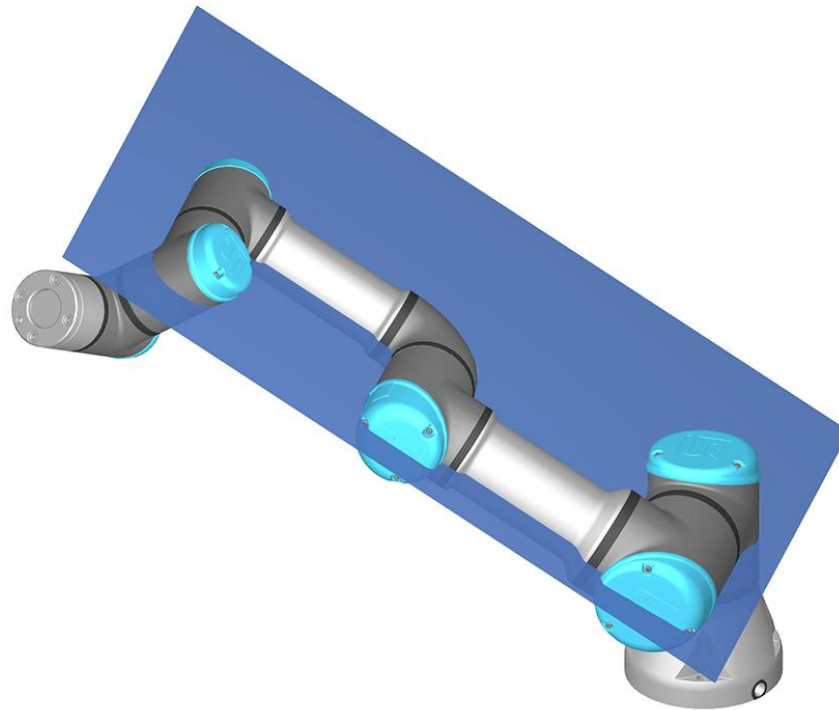
Singulariteti — zašto izbor prostora nije trivijalan

Joint space vs. Task space

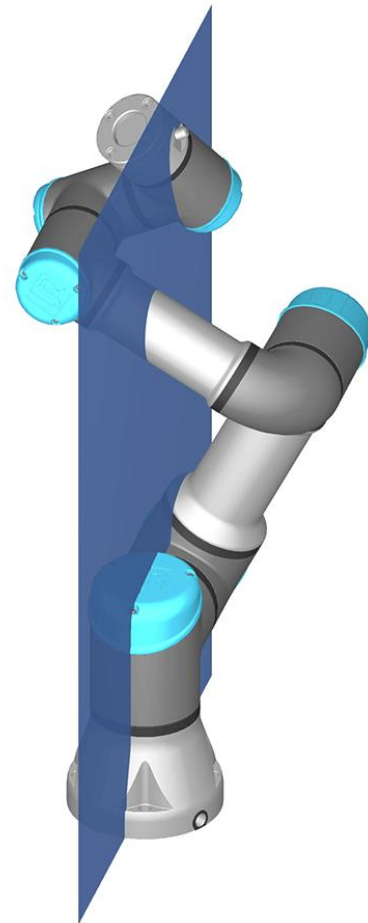
Primjer: singularni položaji UR5 robota



WRIST SINGULARITY



ELBOW SINGULARITY



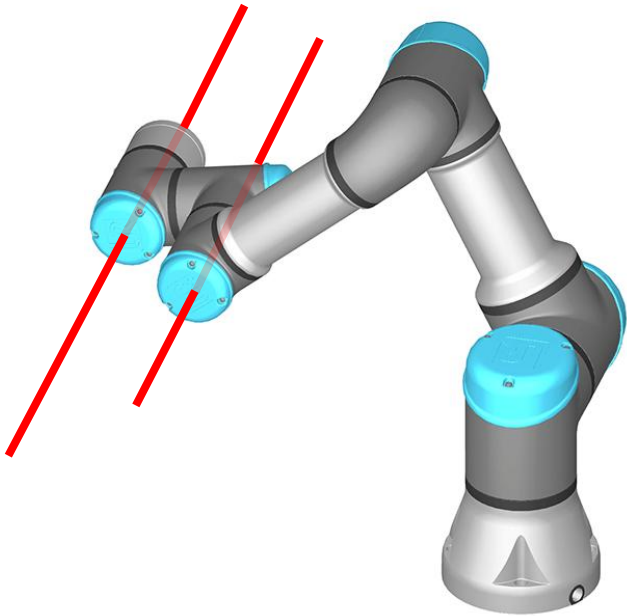
SHOULDER SINGULARITY

Singulariteti – zašto izbor prostora nije trivijalan

Joint space vs. Task space

Primjer: singularni položaji UR5 robota

- Nastaje kad su osi zglobova q_4 i q_6 poravnate, a $q_5 = k180^\circ, k \in \mathbb{Z}$
- Rezultat toga je da su zglobovi q_4 , q_5 , q_6 u istoj ravnini
 - Efektivno, zglobovi q_4 i q_6 rade „isti posao” – izgubili smo jedan rotacijski stupanj slobode gibanja
 - Dolazi do linearne zavisnosti 4. i 6. stupca u Jacobijanu

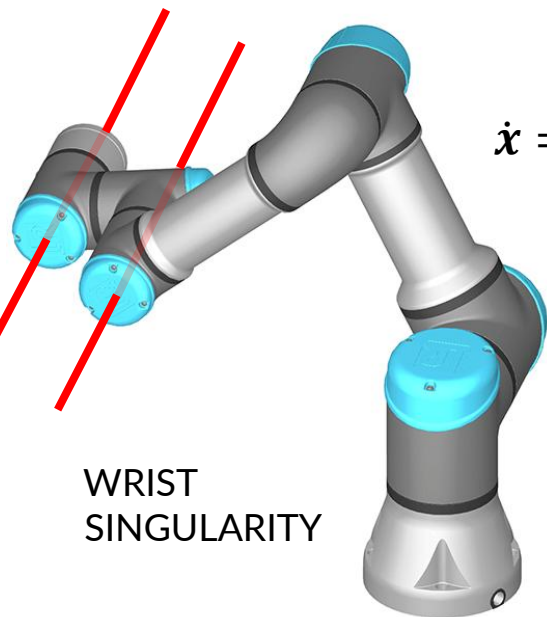


WRIST SINGULARITY

Singulariteti – zašto izbor prostora nije trivijalan

Joint space vs. Task space

Primjer: singularni položaji UR5 robota



Možemo razdvojiti translacijske

i rotacijske brzine

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}} = \begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} \rightarrow \mathbf{J} = \begin{bmatrix} \mathbf{J}_v \\ \mathbf{J}_\omega \end{bmatrix}, \text{ a znamo da vrijedi: } \mathbf{J}(\mathbf{q}) = \frac{\partial \mathbf{f}(\mathbf{q})}{\partial \mathbf{q}}$$

Jacobian je matrica **parcijalnih derivacija** koja lokalno povezuje promjene unutarnjih koordinata robota $\mathbf{q}$ s promjenama vanjskih koordinata $\mathbf{x}$.

Što pojedini element matrice govori?

$$\mathbf{J}_{\omega, wrist} = \begin{bmatrix} \frac{\partial \omega_x}{\partial \dot{q}_4} & \frac{\partial \omega_x}{\partial \dot{q}_5} & \frac{\partial \omega_x}{\partial \dot{q}_6} \\ \frac{\partial \omega_y}{\partial \dot{q}_4} & \frac{\partial \omega_y}{\partial \dot{q}_5} & \frac{\partial \omega_y}{\partial \dot{q}_6} \\ \frac{\partial \omega_z}{\partial \dot{q}_4} & \frac{\partial \omega_z}{\partial \dot{q}_5} & \frac{\partial \omega_z}{\partial \dot{q}_6} \end{bmatrix}$$

Rotacijski dio Jacobiana za wrist

Za pojednostavljeni prikaz orijentacije ruke

možemo uvesti efektivni kut $\boldsymbol{\theta} = q_2, q_3, q_4$

$$\mathbf{J}_{\omega, wrist} = \begin{bmatrix} \sin q_1 & \cos q_1 \sin \boldsymbol{\theta} & \sin q_1 \cos q_5 - \cos q_1 \cos \boldsymbol{\theta} \sin q_5 \\ -\cos q_1 & \sin q_1 \sin \boldsymbol{\theta} & -\cos q_1 \cos q_5 - \sin q_1 \cos \boldsymbol{\theta} \sin q_5 \\ 0 & -\cos \boldsymbol{\theta} & -\sin \boldsymbol{\theta} \sin q_5 \end{bmatrix}$$

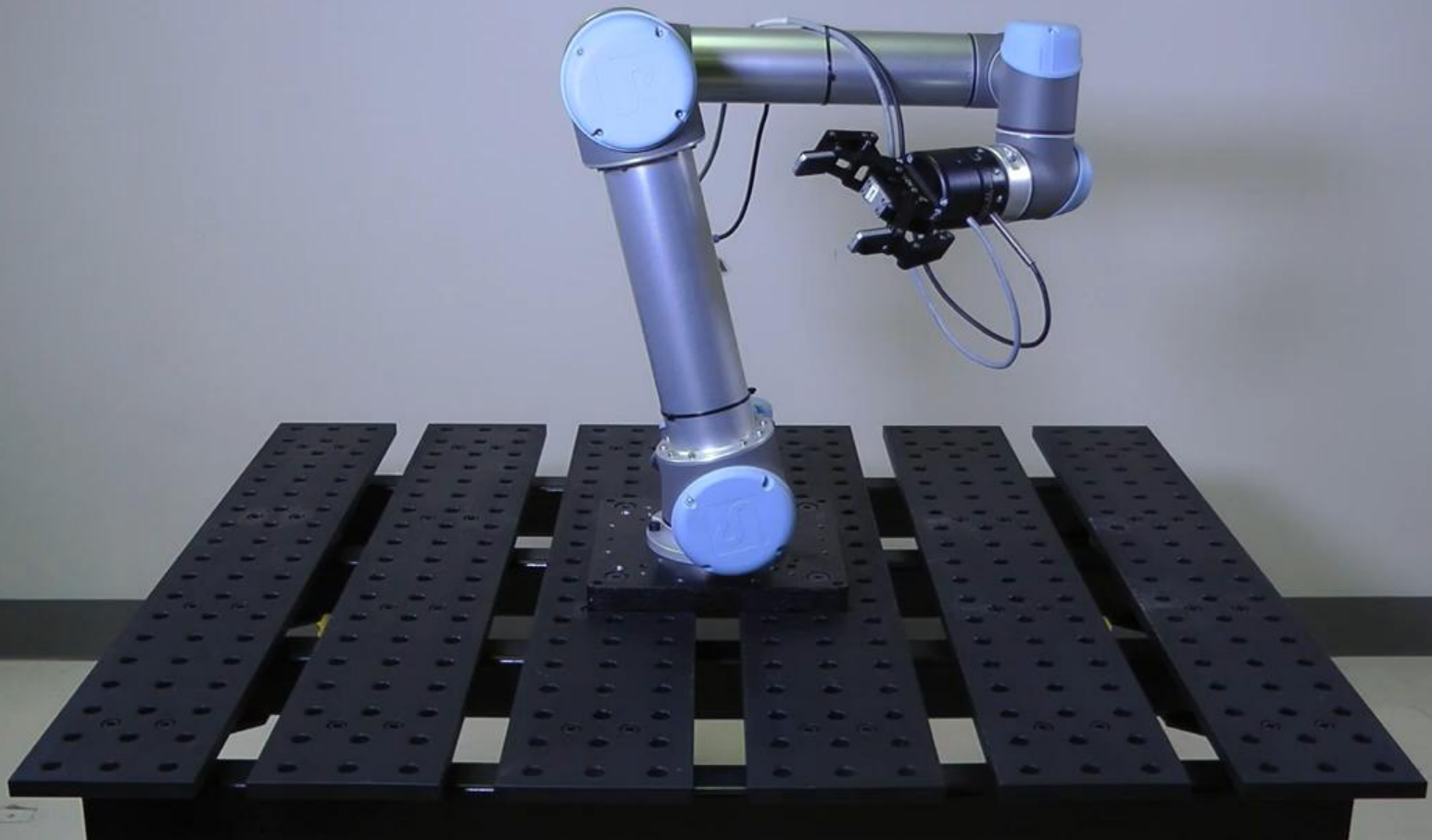
$$\mathbf{J}_{\omega, wrist} = \begin{bmatrix} \boxed{\sin q_1} & \cos q_1 \sin \boldsymbol{\theta} & \boxed{\sin q_1} \\ -\cos q_1 & \sin q_1 \sin \boldsymbol{\theta} & -\cos q_1 \\ \boxed{0} & -\cos \boldsymbol{\theta} & \boxed{0} \end{bmatrix}$$

$\text{rank}(\mathbf{J}_{\omega, wrist}) < 3 \Rightarrow \text{rank}(\mathbf{J}) < 6 \Rightarrow \text{SINGULARITET}$

$$q_5 = 0^\circ \text{ ili } q_5 = 180^\circ \rightarrow \begin{aligned} \sin q_5 &= 0 \\ \cos q_5 &= \pm 1 \end{aligned}$$

Singulariteti — zašto izbor prostora nije trivijalan

Joint vs. Task space command



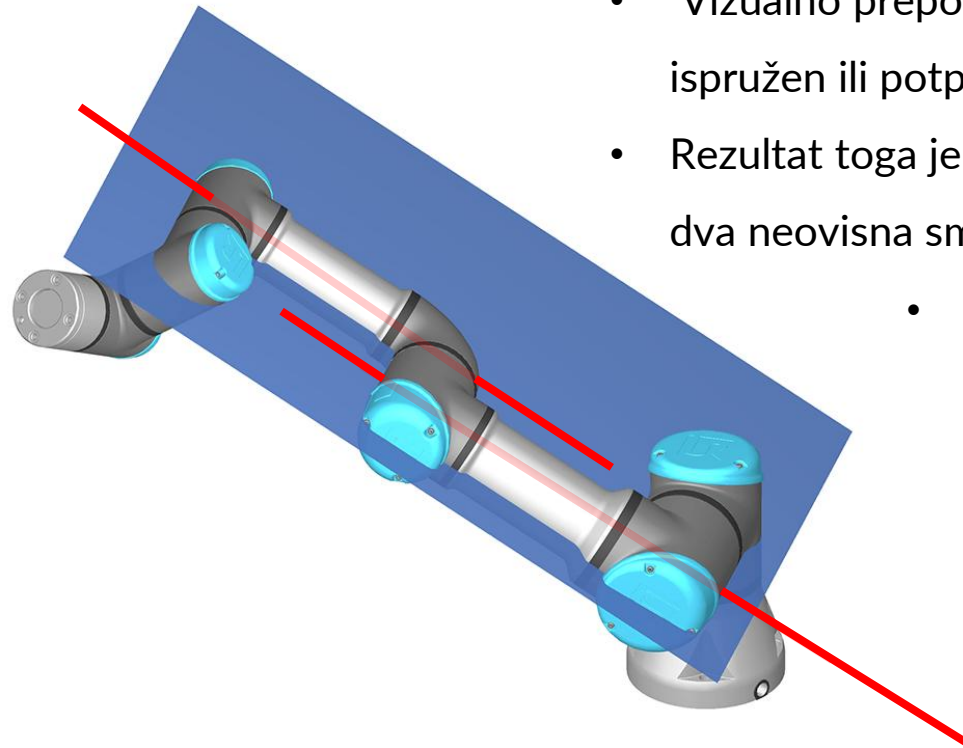
Singulariteti – zašto izbor prostora nije trivijalan

Joint space vs. Task space

Primjer: singularni položaji UR5 robota

- Nastaje kada su drugi i treći link robota kolinearni, a $q_3 = 0$
- Vizualno prepoznamo jer izgleda kao da je robot je potpuno ispružen ili potpuno **sklopljen (?)**
- Rezultat toga je da shoulder i elbow joint više ne proizvode dva neovisna smjera gibanja TCP-a
 - Stupci Jacobiana povezani sa q_2 i q_3 postaju linearno zavisni

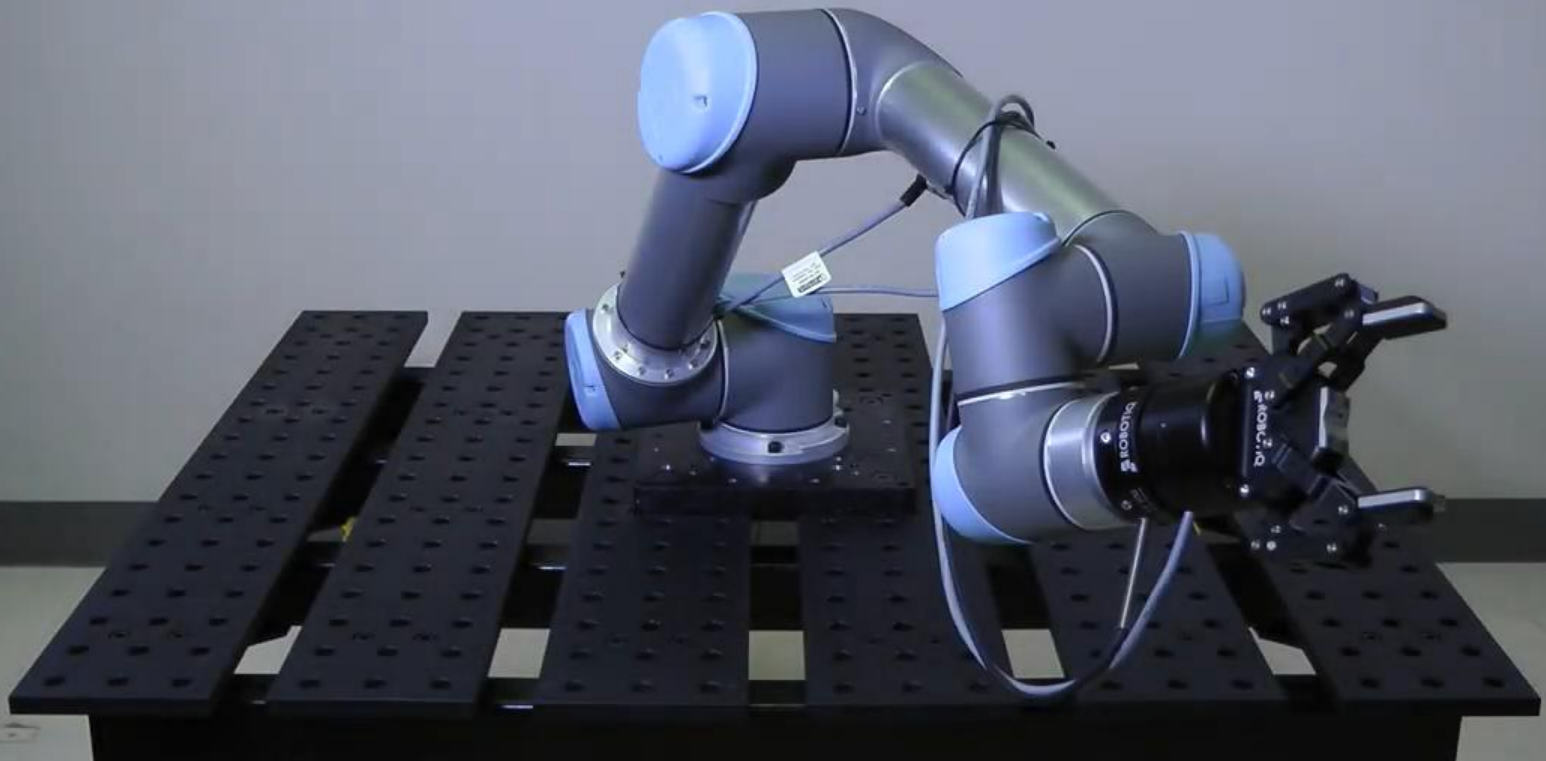
$$c_2 \parallel c_3 \Rightarrow \text{rank}(J) < 6 \Rightarrow \textit{SINGULARITET}$$



ELBOW SINGULARITY

Singulariteti — zašto izbor prostora nije trivijalan

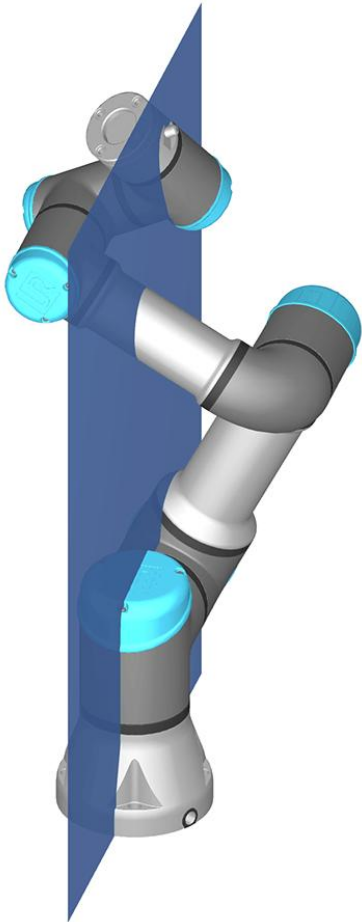
Joint vs. Task space command



Singulariteti – zašto izbor prostora nije trivijalan

Joint space vs. Task space

Primjer: singularni položaji UR5 robota



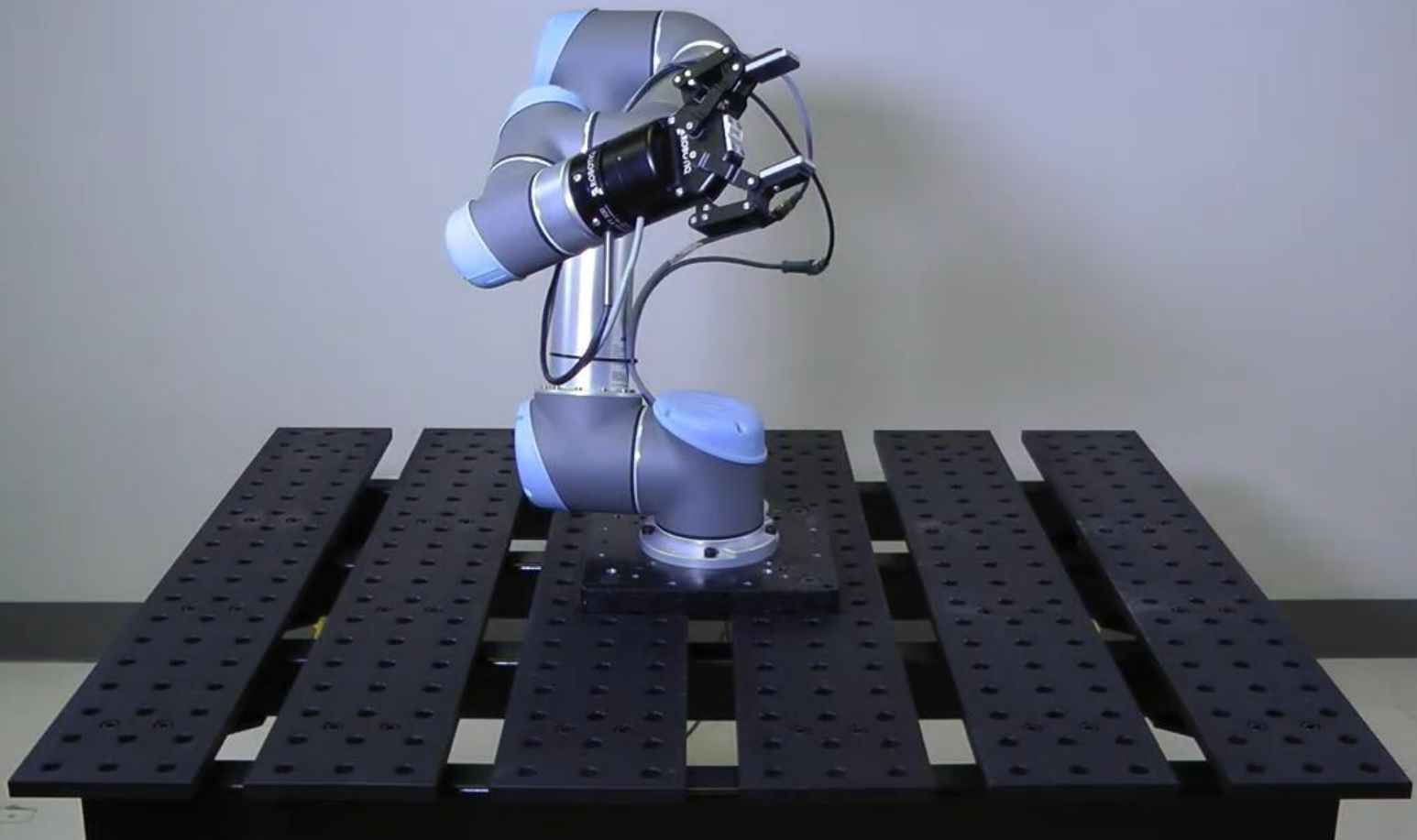
- Nastaje kad sjecište osi zglobova q_5 i q_6 (**wrist centre**) leži na osi zgloba q_1
- Rotacija zgloba q_1 više ne uzrokuje translacijsko gibanje wrist centra, već samo rotaciju oko osi baze
- Prvi stupac Jacobijana gubi neovisnost, pa rang matrice opada

$$\text{rank}(\mathbf{J}) < 6 \Rightarrow \textit{SINGULARITET}$$

SHOULDER SINGULARITY

Singulariteti — zašto izbor prostora nije trivijalan

Joint vs. Task space command



Što s robotima koji su redundantne kinematike?

Franka Emika
Panda



Valkyrie



Prednosti: 7 DOF omogućuje veću fleksibilnost i prirodnije pokrete zahvaljujući redundantnosti.
Mane: 7 DOF povećava složenost upravljanja i zahtijeva naprednije algoritme.



iCub



Unitree G1

Singulariteti kod redundantnih kinematičkih struktura

- Kad je J kvadratna i punog ranga, rješenje je jedinstveno: $\dot{q} = J^{-1}(q) \dot{x}$
- Redundantni manipulator (npr. **7 DOF**) nema kvadratnu matricu nego je 6×7 – sustav $\dot{x} = J(q)\dot{q}$ ima beskonačno rješenja, ne možemo dobiti inverz
- Uvodimo **PSEUDOINVERZ** (J^+ ili J^+ ili $J^\#$) – bira jedno specifično rješenje od tih beskonačnih
- Najčešći je Moore-Penrose pseudoinverz koji bira jedinstveno rješenje s **minimalnom normom** $\|\dot{q}\|$ – najmanje naprezanje motora

- Opće rješenje za redundantni slučaj:**

Gdje je I jedinična matrica, a je z vektor koji određuje kojim intenzitetom i u kojem smjeru se zglobovi gibaju unutar null-spacea

$$\dot{q} = \underbrace{J^+}_{\substack{\text{Praćenje} \\ \text{putanje TCP-a}}} \dot{x} + \underbrace{(I - J^+ J)}_{\substack{\text{Null-space} \\ \text{pretraživanje} \\ \text{i optimizacija}}} z$$

$\begin{matrix} 7 \times 1 & 7 \times 6 & 6 \times 1 & & 7 \times 7 & 7 \times 6 & 6 \times 7 & 7 \times 1 \\ \hline \end{matrix}$

- Null-space** = slobodni prostor brzina zglobova u kojem se zglobovi mogu gibati bez ikakvog efekta na gibanje TCP-a (${}^{baza}T_{TCP} - \text{const.}$)

- Null-space $\neq$ singularitet** – npr. za $J \in \mathbb{R}^{6 \times 7}$

- Redundantnost: $\text{rank}(J) = 6, \dim(\mathcal{N}(J)) = 1$
- Singularitet: $\text{rank}(J) < 6, \dim(\mathcal{N}(J)) = 7 - \text{rank}(J)$

Redundantnost ne uklanja singularitete, ali daje robotu slobodu da ih aktivno izbjegava pretraživanjem null-spacea

Izračun pseudoinverza – SVD

- **SVD** (engl. *Singular Value Decomposition*) je metoda kojom bilo koju matricu možemo prikazati kao umnožak tri matrice, pa Jacobi matricu možemo zapisati kao: $J = U\Sigma V^T$
 1. U – ortogonalna matrica čiji stupci su smjerovi kretanja u **task spaceu** (kako se end-effector giba). $\text{Dim}(U) = 6 \times 6$
 2. Σ – dijagonalna matrica sa **singularnim vrijednostima** σ_i na dijagonali – govore koliko učinkovito joint gibanje proizvodi TCP gibanje u tom smjeru. $\text{Dim}(\Sigma) = 6 \times 7$
 3. V^T – ortogonalna matrica čiji stupci su smjerovi kretanja u **joint spaceu** (kako se zglobovi gibaju). $\text{Dim}(V) = 7 \times 7$
- **Matrica Σ** – veliki iznosi σ_i znače da se robot lako kreće u tom smjeru, mali iznosi σ_i znače da male joint brzine daju veliku TCP brzinu u tom smjeru ($\sigma_i \rightarrow 0$, singularitet)

$$J^+ = V \underbrace{\Sigma^+}_{\text{invertirani } \Sigma} U^T \rightarrow \text{Moore-Penrose pseudoinverz}$$

pravilom: svaku nenultu σ_i zamijeni se s $\frac{1}{\sigma_i}$, nule ostaju nule. $\text{Dim}(\Sigma^+) = 7 \times 6$

Izračun pseudoinverza – regularizacija

$J^+ = V \Sigma^+ U^T$ Problem nastaje pri računanju Σ^+ , jer za male σ_i član $\frac{1}{\sigma_i}$ divergira

Kada $\sigma_i \rightarrow 0$, $\frac{1}{\sigma_i}$ teži u beskonačnost $\longrightarrow$ Analogno klasičnom singularitetu Jacobiana, brzine zglobova „eksplodiraju”

- Rješenje je Damped Least Squares (Levenberg-Marquardt regularizacija),
umjesto $\frac{1}{\sigma_i}$ koristi se $\frac{\sigma_i}{\sigma_i^2 + \lambda^2}$, gdje je λ mali broj (tipično se kreće između 0.001 i 0.1, ovisno o robotu i aplikaciji – nije fiksni parametar)
- Moguća su dva ishoda:
 1. σ_i je velik broj (daleko od singulariteta): $\frac{\sigma_i}{\sigma_i^2 + \lambda^2} \approx \frac{1}{\sigma_i}$
 - Regularizacija nema utjecaj
 2. σ_i je mali broj (blizu singulariteta): $\frac{\sigma_i}{\sigma_i^2 + \lambda^2} \rightarrow 0$
 - Problematici smjer se prigušuje
 - Joint brzine ostaju ograničene

Nijedan član više ne divergira prema ∞ , pa numeričko ponašanje ostaje stabilno

Singulariteti kod redundantnih kinematičkih struktura

1. Uvodimo pseudoinverz kako bi mogli izračunati potrebne brzine zglobova za željene brzine TCP-a:

$$\dot{\mathbf{q}} = \mathbf{J}^{\dagger} \dot{\mathbf{x}}$$

2. Pseudoinverz se može definirati SVD-om, a za puni row-rank slučaj uvijek vrijedi:

$$\dot{\mathbf{q}} = \mathbf{J}^T (\mathbf{J}\mathbf{J}^T)^{-1} \dot{\mathbf{x}}$$

3. Kako bi stabilno računali pseudoinverz i zadržali razumne joint brzine kada smo blizu singulariteta, uvodimo Levenberg-Marquardt regularizaciju:

$$\dot{\mathbf{q}} = \mathbf{J}^T (\mathbf{J}\mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \dot{\mathbf{x}}$$

4. Optimizaciju null-spacea uvodimo kako bi aktivno udaljavali robota od singularnih konfiguracija i iskorištavali redundantnost:

$$\dot{\mathbf{q}} = \mathbf{J}^T (\mathbf{J}\mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \dot{\mathbf{x}} + (\mathbf{I} - \mathbf{J}^T (\mathbf{J}\mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \mathbf{J}) \mathbf{z}$$

SVAKI ROBOT IMA SINGULARNE POLOŽAJE!!!

Joint space vs Task space

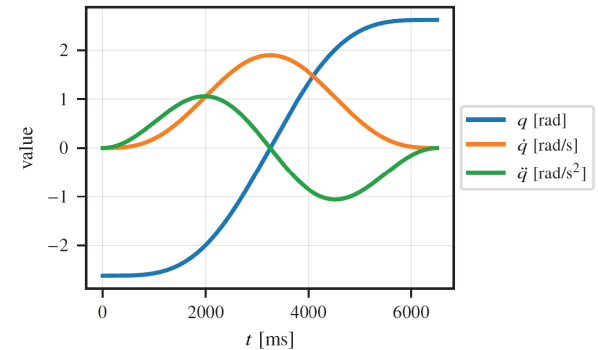
- Trajektorijama u joint space-u dajemo prednost zbog toga što:
 - Nema kompleksnog računanja (kinematički modeli)
 - Trajektorije nisu pod utjecajem singularnih konfiguracija
 - Limiti (pozicija, brzina i momenata) na svakom zglobu su nam unaprijed poznati (datasheet)
- S druge strane, **task space** je intuitivniji za definiranje zadatka u radnom prostoru (točka A $\rightarrow$ točka B, izbjegavanje prepreka)
- **Redundantne kinematičke strukture** omogućuju najbolje od oba pristupa:
 - praćenje zadane putanje TCP-a
 - dodatni stupnjevi slobode koriste se za izbjegavanje singulariteta, limita zglobova i optimizaciju potrošnje energije
- Kod redundantnih robota singularitet se može zaobići prekonfiguracijom kroz null-space, dok se kod neredundantnih robota singularna područja uglavnom izbjegavaju promjenom putanje ili orijentacije TCP-a

Što je dobra trajektorija? – kriteriji kvalitete

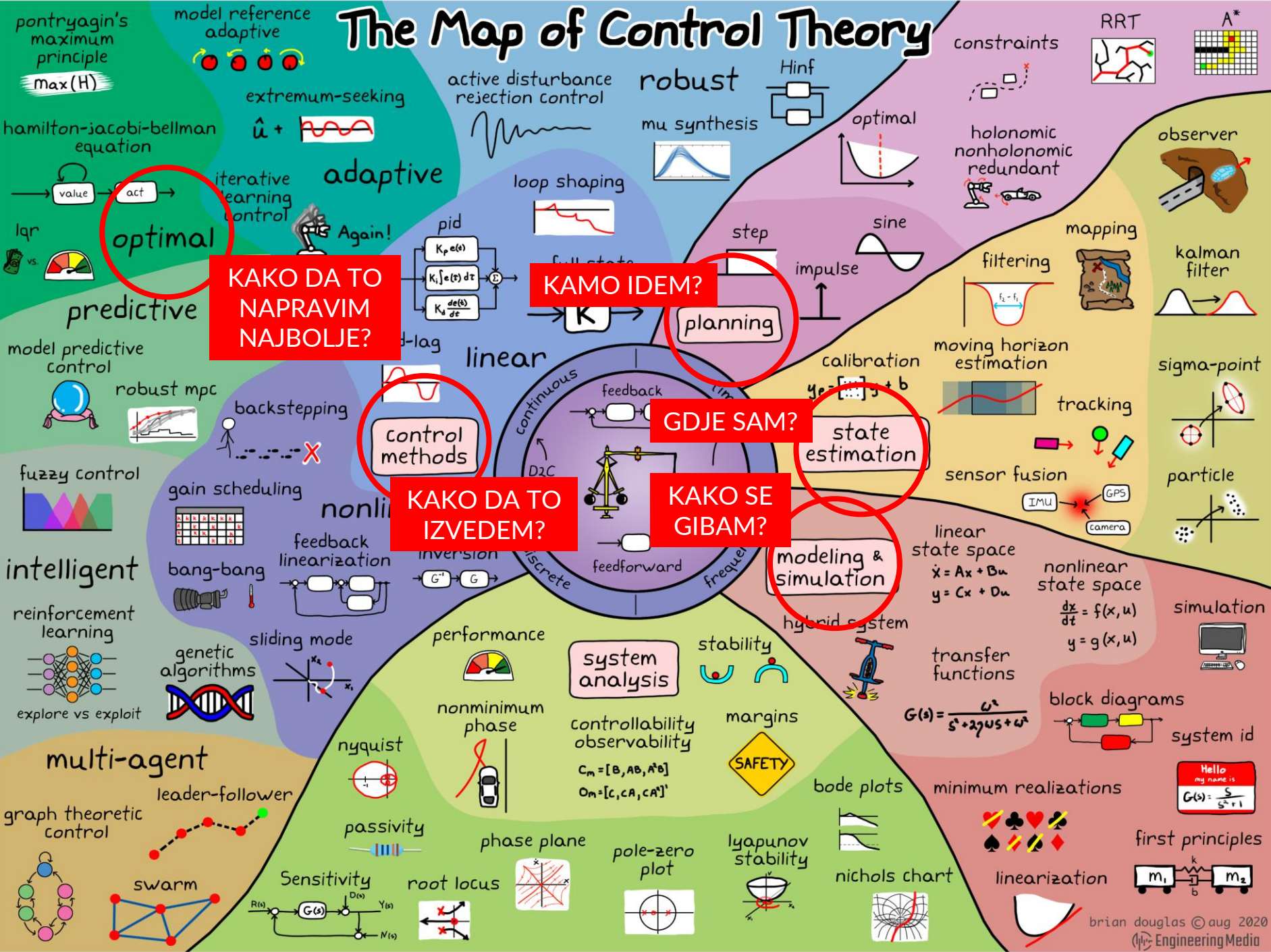
1. **Glatkoća** - viši stupanj kontinuiranosti smanjuje vibracije i olakšava upravljanje (C^2 kontinuiranost)
2. **Limiti** - trajektorija mora poštivati hardverska ograničenja aktuatora i zglobova
3. **Trzaj** - minimizacija trzaja povećava trajnost sustava i kvalitetu gibanja
4. **Izvedivost** - trajektorija ne smije zahtijevati nedostižne konfiguracije ili singularne prolaze
5. **Vrijeme** - često se traži kompromis između brzine izvršenja i kvalitete gibanja

C^2 kontinuiranost trajektorija:

- Trajektoriju opisujemo funkcijom položaja: $q(t)$
- Njene derivacije su:
 - Brzina: $\dot{q}(t)$
 - Akceleracija: $\ddot{q}(t)$
 - Trzaj (jerk): $\dddot{q}(t)$
- C^2 kontinuiranost znači da su položaj, brzina i akceleracija kontinuirani, odnosno bez skokova
- Zašto to želimo kod robota?
 - Mirnije gibanje → lakše praćenje upravljačkim sustavom
 - Manje vibracija i udara → manje trošenje mehanike i motora
 - Prirodnije i sigurnije gibanje robota (HRI)



The Map of Control Theory



Kako opisujemo gibanje robota?

MATEMATIČKI MODELI TRAJEKTORIJA

Point-to-Point Joint Space planiranje trajektorija

Zašto Joint-Space Planning?

- Robot često mora prijeći iz jedne konfiguracije zglobova u drugu: $q^i \rightarrow q^f$
- Naivno pomicanje zglobova “direktno do cilja” nije dovoljno dobro, jer gibanje mora biti:
 - glatko
 - brzo, ali sigurno
 - unutar ograničenja aktuatora
 - ponovljivo i stabilno za upravljanje
- Zašto planirati u joint space-u? U joint prostoru izravno upravljamo zglobovima robota, pa su ograničenja prirodno definirana
- Umjesto da samo znamo **gdje robot treba završiti**, želimo znati **kako tamo doći kvalitetno**

Point-to-Point Joint Space planiranje trajektorija

Matematički model trajektorije

- Trajektoriju svakog zgloba opisujemo funkcijom vremena: $\mathbf{q}(t)$
- Tražimo funkciju koja vodi robot iz početnog u završni položaj:

$$\mathbf{q}(0) = \mathbf{q}^i, \mathbf{q}(t_f) = \mathbf{q}^f$$

- Standardni oblik interpolacije: $\mathbf{q}(t) = \mathbf{q}^i + \underbrace{s(t)}_{\substack{\text{Normalizirana} \\ \text{funkcija gibanja} \\ s(t) \in [0,1]}} \underbrace{(\mathbf{q}^f - \mathbf{q}^i)}_{\substack{\text{Ukupni} \\ \text{pomak } D}}$

- Oblik funkcije $s(t)$ određuje ponašanje gibanja
- Cijeli problem planiranja ptp često se svodi na odabir dovoljno dobre funkcije $s(t)$

Što ona određuje?

- Brzinu, akceleraciju, trzaj i ukupnu kvalitetu gibanja

Point-to-Point Joint Space planiranje trajektorija

Matematički model trajektorije

- Kako bi zadovoljili **C^2 kontinuiranost**, potrebno je zadovoljiti sljedeće rubne uvjete:
 1. Početni položaj - $s(0)$
 2. Završni položaj - $s(t_f)$
 3. Početna brzina - $\dot{s}(0)$
 4. Završna brzina - $\dot{s}(t_f)$
 5. Početna akceleracija - $\ddot{s}(0)$
 6. Završna akceleracija - $\ddot{s}(t_f)$
- **Rubni uvjeti** definiraju **putanju** ($q(s)$), a ne **trajektoriju** ($q(t)$)
- Trajektorija nastaje tek odabirom **vremenskog skaliranja** $s(t)$
- **Opće pravilo:** broj koeficijenata = maksimalan broj rubnih uvjeta koje možemo zadovoljiti

Point-to-Point Joint Space planiranje trajektorija

Izbor funkcije $s(t)$: Interpolacija polinomom prvog, trećeg i petog reda

1. Interpolacija polinomom 1. reda

$$s(t) = a_0 + a_1 t$$

Moramo zadovoljiti rubne uvjete: $s(0) = 0$, $s(t_f) = 1$

Rješenje: $a_0 = 0$ $a_1 t_f = 1 \rightarrow a_1 = \frac{1}{t_f}$

$$s(t) = \frac{t}{t_f} = \tau$$

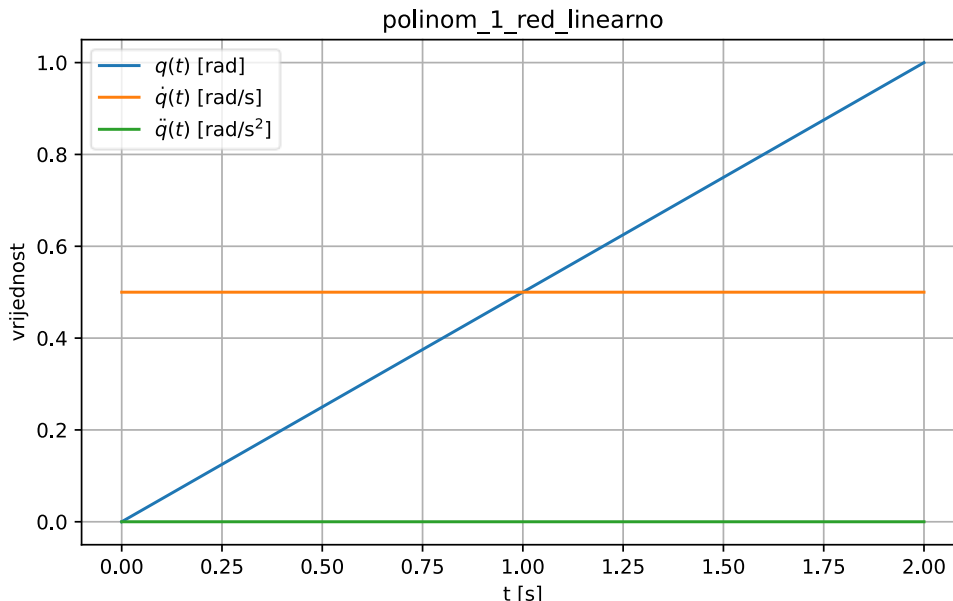
Uvodimo **normalizirano vrijeme** τ
(predstavlja napredak po trajektoriji
neovisno o njenom trajanju)

Minimalno dopušteno vrijeme s obzirom na
ograničenje brzine:

$$s(t) = \frac{t}{t_f} \quad \dot{s}(t) = \frac{1}{t_f} \quad \ddot{s}(t) = 0$$

$$|\dot{q}|_{\max} = \frac{|D|}{t_f}, \text{ uz uvjet } |q(\dot{t})| \leq \dot{q}_{\max} :$$

$$\frac{|D|}{t_f} \leq \dot{q}_{\max} \rightarrow t_f \geq \frac{|D|}{\dot{q}_{\max}}$$



Point-to-Point Joint Space planiranje trajektorija

Izbor funkcije $s(t)$: Interpolacija polinomom prvog, trećeg i petog reda

2. Interpolacija polinomom 3. reda

$$s(t) = a_0 + a_1t + a_2t^2 + a_3t^3$$

Moramo zadovoljiti rubne uvjete: $s(0) = 0, s(t_f) = 1 \rightarrow$ Moramo proći 100% trajektorije
 $\dot{s}(0) = 0, \dot{s}(t_f) = 0 \rightarrow$ Želimo da robot krene i završi s nulom brzine

Rješenje:

Za slučaj $t = 0; s(0) = a_0 = 0$

$$\dot{s}(t) = a_1 + 2a_2t + 3a_3t^2$$

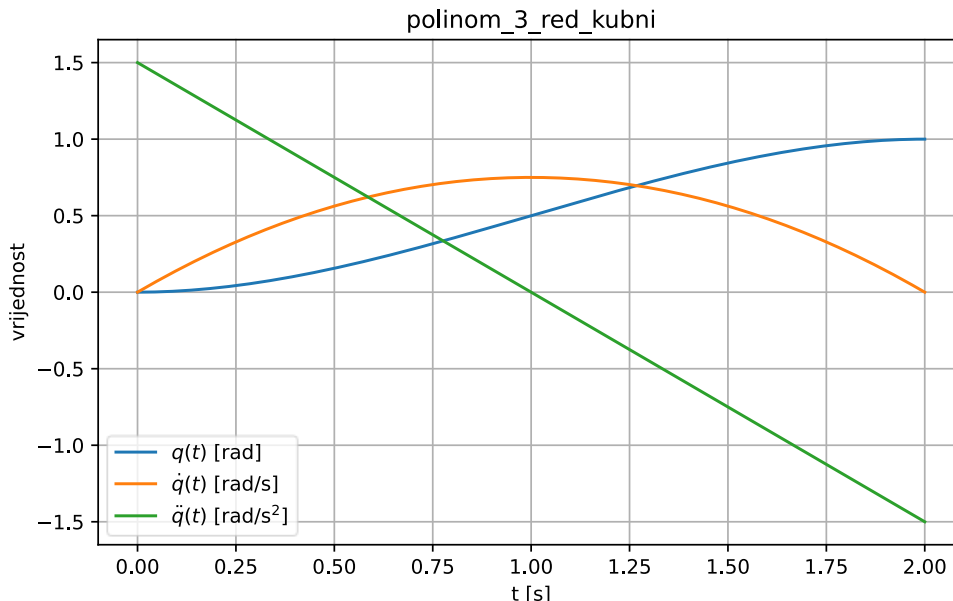
Za slučaj $t = 0; \dot{s}(0) = a_1 = 0$

$$s(t_f) = a_2t_f^2 + a_3t_f^3 = 1$$

$$\dot{s}(t_f) = 2a_2t_f + 3a_3t_f^2 = 0$$

$$a_2 = \frac{3}{t_f^2}, a_3 = \frac{-2}{t_f^3}$$

$$s(t) = 3\tau^2 - 2\tau^3$$



Point-to-Point Joint Space planiranje trajektorija

Izbor funkcije $s(t)$: Interpolacija polinomom prvog, trećeg i petog reda

3. Interpolacija polinomom 5. reda

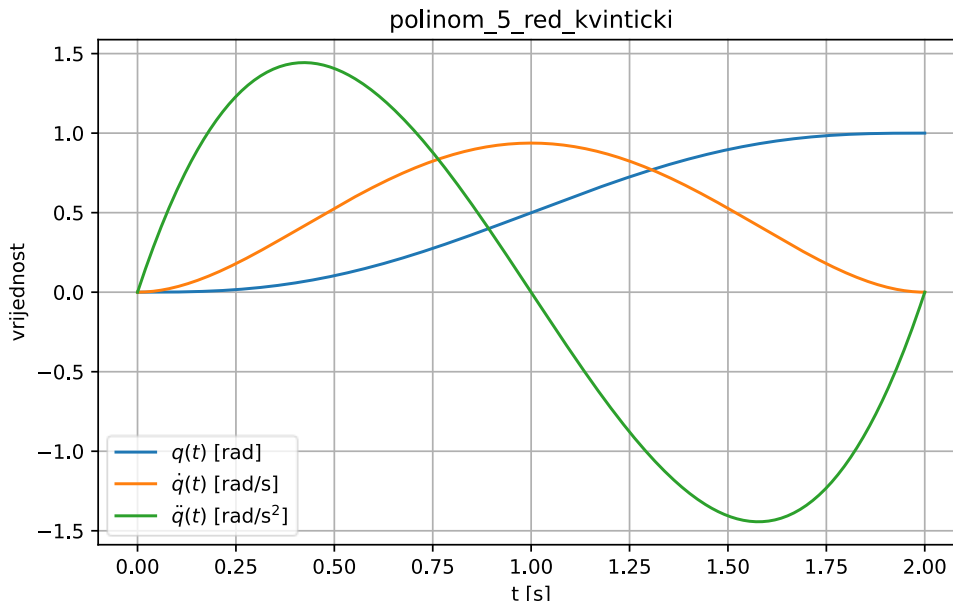
$$s(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

Moramo zadovoljiti rubne uvjete: $s(0) = 0, s(t_f) = 1 \rightarrow$ Moramo proći 100% trajektorije

$\dot{s}(0) = 0, \dot{s}(t_f) = 0 \rightarrow$ Želimo da robot krene i završi s

želim da robot krene i završi s $\leftarrow \ddot{s}(0) = 0, \ddot{s}(t_f) = 0$ nulom brzine

nulom akceleracije



$$s(t) = 10\tau^3 - 15\tau^4 + 6\tau^5$$

$$\dot{s}(t) = \frac{1}{t_f} (30\tau^2 - 60\tau^3 + 30\tau^4)$$

$$\ddot{s}(t) = \frac{1}{t_f^2} (60\tau - 180\tau^2 + 120\tau^3)$$

C² kontinuiranost $\rightarrow$ glatko gibanje bez naglih promjena akceleracije

Point-to-Point Joint Space planiranje trajektorija

Minimalno dopušteno vrijeme s obzirom na ograničenje brzine

2. Interpolacija polinomom 3. reda

$$s(t) = 3\tau^2 - 2\tau^3$$

$$t_f \geq t_{vel}$$

$$t_f \geq t_{acc}$$

$$t_f = \max(t_{vel}, t_{acc})$$

$$t_{vel} = \frac{3|D|}{2\dot{q}_{max}}$$

$$t_{acc} = \sqrt{\frac{6D}{\ddot{q}_{max}}}$$

Zašto gledamo i akceleraciju?

3. Interpolacija polinomom 5. reda

$$s(t) = 10\tau^3 - 15\tau^4 + 6\tau^5$$

$$t_f \geq t_{vel}$$

$$t_f \geq t_{acc}$$

$$t_f = \max(t_{vel}, t_{acc})$$

$$t_{vel} = \frac{1.875|D|}{\dot{q}_{max}}$$

$$t_{acc} = \sqrt{\frac{5.77D}{\ddot{q}_{max}}}$$

Kod polinoma 5. reda akceleracija često postaje dominantno ograničenje

Usporedba interpolacije polinomom prvog, trećeg i petog reda:

Red polinoma	Broj uvjeta	Kontinuirano	Koristi se	Primjena
1	položaj	C^0	rijetko	Pomak osi robota iz A u B bez brige o dinamici (offline simulacija, grubo planiranje)
3	položaj + brzina	C^1	često	Pick and place (robot uzme objekt i prebaci ga na drugu lokaciju)
5	položaj + brzina + akceleracija	C^2	standard	Precizna robotika (kontaktna – s okolinom, ljudima), klasična identifikacija sustava

Point-to-Point Task Space planiranje trajektorija

Definicija

- Želimo pomaknuti TCP robota transformacijom $T^i \rightarrow T^f$, koja sadrži poziciju p i orijentaciju A
- To su zapravo dva problema, pa ih razdvajamo:
 - Translacija (pozicija): $p(t) = p_i + s(t)(p_f - p_i)$
 - Rotacija (orijentacija): slerp
- Na taj način problem postaje dva 1D problema:
 - Translacija: pomičemo se po pravcu između dvije točke
 - Rotacija: rotiramo oko osi u za kut koji se “dozira” preko $s(t)$
- Za oboje koristimo isti $s(t)$ jer na taj način osigurava da translacija i rotacija napreduju istim tempom, čime se dobiva glatko i koordinirano gibanje TCP-a

Point-to-Point Task Space planiranje trajektorija

Kako iz task-space u joint-space?

- Ključni problem:
 - Trajektoriju definiramo u task space-u: $p(t), R(t)$
 - Robotom upravljamo u joint space-u: $q(t)$

} Treba nam preslikavanje:
task $\rightarrow$ joint space
Rješenje?
- Inverzna kinematika:
 - Za svaki trenutak t moramo odrediti: $q(t) = IK(p(r), R(t))$
 - 3 glavna pristupa:
 1. Analitička inverzna kinematika – iz kinematike robota
 2. Numerička inverzna kinematika – preko Jacobijana iterativno
 3. IK u kontroleru robota – zadajemo samo referencu
- Važne napomene:
 - Inverzna kinematika može imati više rješenja ili nijedno rješenje
 - Potrebno je osigurati kontinuitet rješenja (C^2) i izbjegavati singularitete

Point-to-Point Task Space planiranje trajektorija

Odabir vremena t_f u task-space-u

- Vrijeme trajektorije određujemo prema maksimalnoj brzini TCP-a $v_{\max}$ i maksimalnoj kutnoj brzini $\omega_{\max}$

- Translacija: $D = \|p_f - p_i\|$

$$t_{vel} = k_v \frac{D}{v_{\max}}$$

$$t_{acc} = \sqrt{k_a \frac{D}{a_{\max}}}$$

- Rotacija:

α = kut rotacije između R_i i R_f

$$t_{rot} = k_v \cdot \frac{\alpha}{\omega_{\max}}$$

- Ukupno vrijeme (sinkronizacija):

$$t_f = \max(t_{vel}, t_{acc}, t_{rot})$$

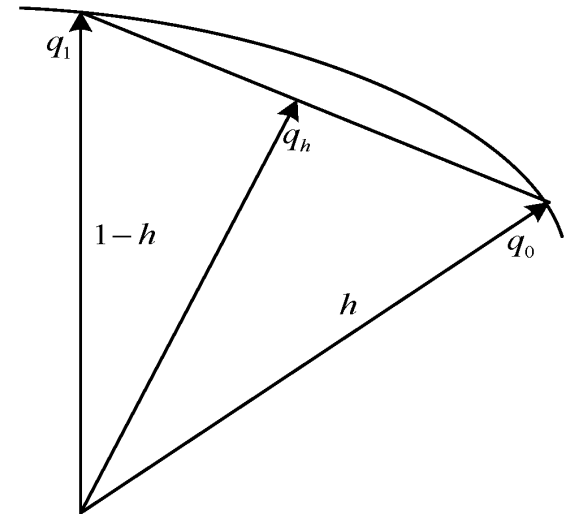
Polinom	Ograničenja	k_v	k_a
1. red	samo brzina	1	–
3. red	brzina + akceleracija	1.5	6
5. red	brzina + akceleracija	1.875	5.77

Point-to-Point Task Space planiranje trajektorija

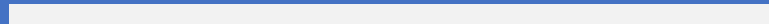
SLERP (engl. *Spherical Linear Interpolation*)

- Koristi se za interpolaciju orijentacije (rotacije) kako bi dobili glatke i konstantne profile kutnih brzina
- Izbjegava probleme linearne interpolacije rotacijskih matrica
- Linearna interpolacija rotacija:
 - nije fizički ispravna
 - može promijeniti brzinu i smjer rotacije

$$R(t) = \text{SLERP} \left(R_i, R_f, s(t) \right)$$

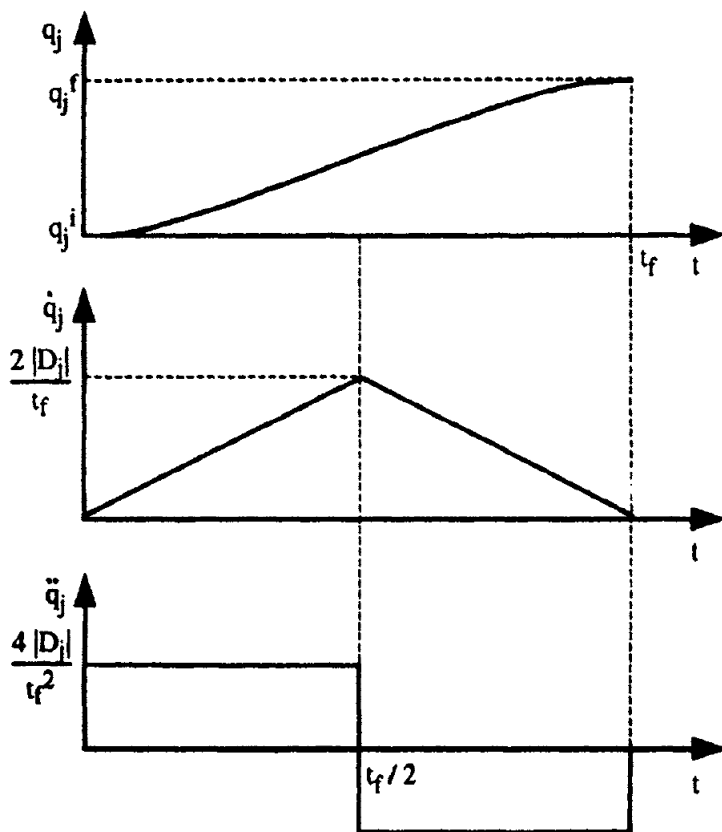


Drugi načini definiranja trajektorija



Bang-bang profil akceleracije

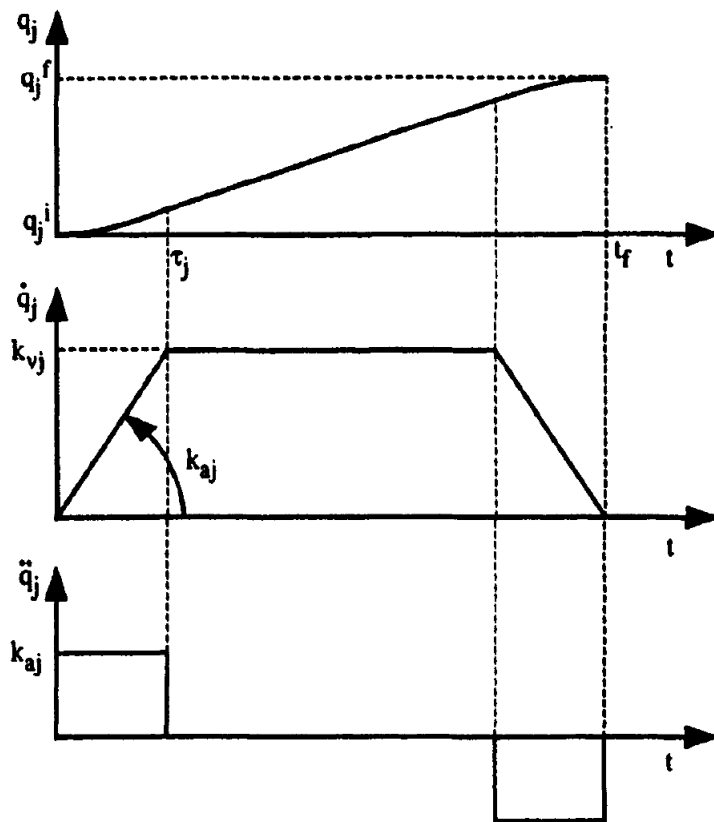
Definiran u joint space-u



Napomena, za sve izraze vezane za generiranje profila u ovoj sekciji proučiti literaturu: Khalil, W., & Dombre, E. (2004). Modeling, identification and control of robots. Butterworth-Heinemann

Trapezni profil brzine

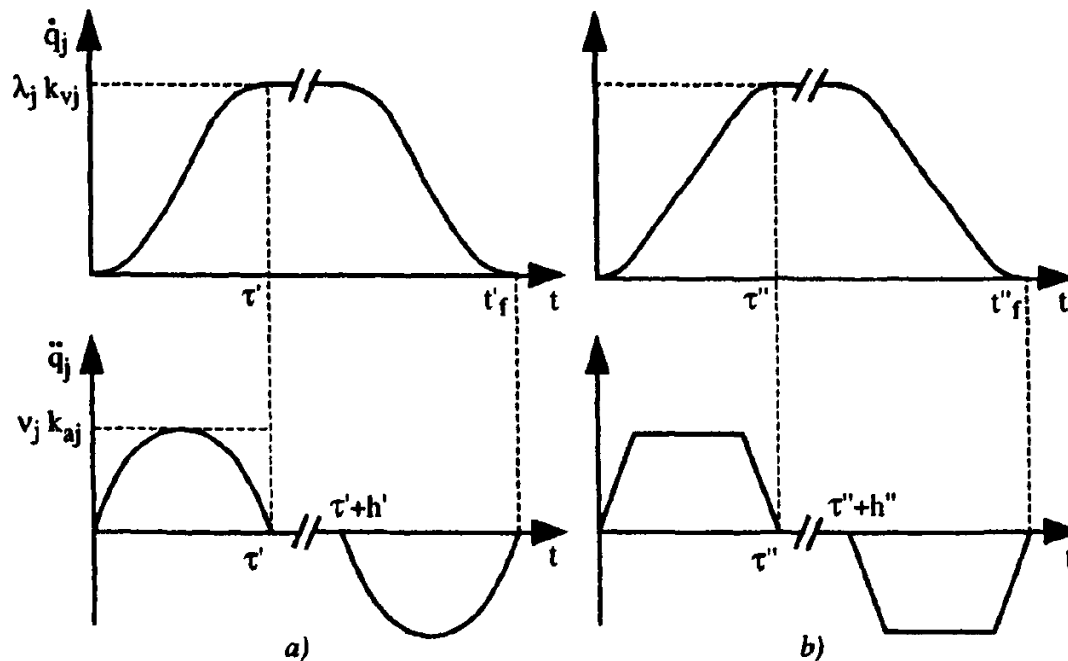
Definiran u joint space-u



Napomena, za sve izraze vezane za generiranje profila u ovoj sekciji proučiti literaturu: Khalil, W., & Dombre, E. (2004). Modeling, identification and control of robots. Butterworth-Heinemann

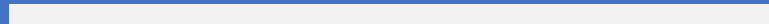
Kontinuirani profil akceleracije s konstantnim profilom brzine

Definiran u joint space-u



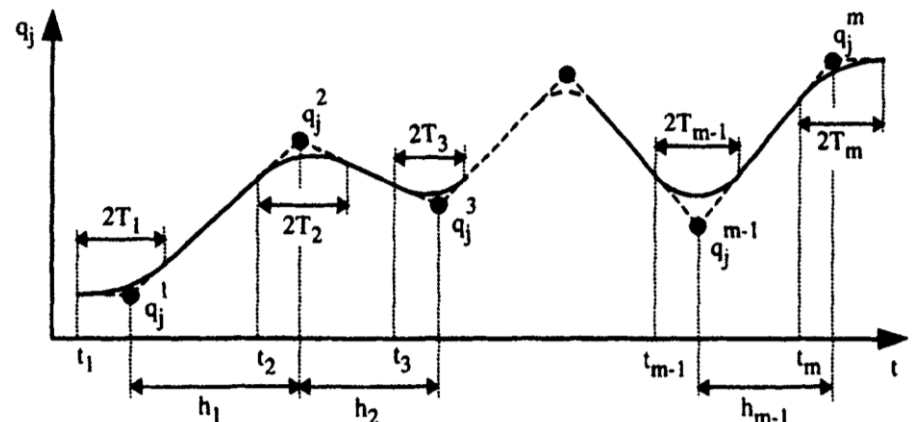
Napomena, za sve izraze vezane za generiranje profila u ovoj sekciji proučiti literaturu: Khalil, W., & Dombre, E. (2004). Modeling, identification and control of robots. Butterworth-Heinemann

Generiranje trajektorija kroz više točaka

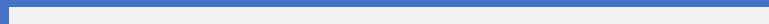


Generiranje trajektorija kroz više točaka

- Do sada: gibanje između dvije točke (point-to-point)
- U praksi: robot mora proći kroz više međutočaka (waypoints)
- Mehanizam:
 - Trajektoriju dijelimo na **segmente**
 - Za svaki segment definiramo interpolaciju $s_i(t)$
 - Između segmenata **MORA POSTOJATI C^2 kontinuitet**
- Dva pristupa:
 - Stop-and-go:
 - robot staje u svakoj točki
 - Smooth trajectory (blending):
 - nema zaustavljanja



Pobudne trajektorije



Što su pobudne trajektorije?

- Posebna vrsta trajektorija koja se koristi za pobudu dinamički svojstava neke kinematičke strukture
- Pobuđivanjem tih svojstava moguće je provesti proces identifikacije dinamičkih parametara robota **Koji su to parametri?**
- Mase segmenata, centri mase svakog segmenta, matrice inercije, trenje u zglobovima, inercijska matrica robota, Coriolisovi i centrifugalni članovi, gravitacijski vektor...

Zašto identificiramo dinamičke parametre robota?

- Dinamički model robota:

$$\tau = H(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q)$$

ovisi o fizičkim parametrima (**mase, centri masa, momenti inercije** – segment)

- **Problem:** stvarne vrijednosti nisu poznate
- **Cilj identifikacije:** dobiti **parametre** koji najbolje opisuju stvarno dinamičko ponašanje

Koraci identifikacije dinamičkih parametara robota:

1. Generiranje pobudnih trajektorija
2. Prikupljanje podataka (eksitacija)
3. Formulacija identifikacijskog problema (optimizacijski problem)
4. Validacija parametara

Generiranje pobudnih trajektorija:

- U literaturi se najčešće koristi **konačna Fourierova serija** (*multisine*)
- Zašto?
 - Analitički izraz položaja, brzine i ubrzanja
 - Kontrola spektra pobude
 - Glatke C^2 trajektorije
 - Izvedivost na stvarnom robotu
 - Velika informativnost trajektorija – mjerljivo!

Generiranje pobudnih trajektorija:

Naše istraživanje – definicija parametara

- Vrijednosti parametara **nisu odabrane**, **izvedene** su iz **fizičkih ograničenja robota** (posebno za svaki zglob)

$$\dot{q}_i(\boxed{t}) = \frac{4\boxed{B_i}}{3} \sin^3(\boxed{wt})$$

Diagram illustrating the parameters of the equation:

- UKUPNO TRAJANJE** (Total Duration) points to the time variable t .
- AMPLITUDA** (Amplitude) points to the amplitude parameter B_i .
- FREKVENCIJA** (Frequency) points to the frequency parameter w .

- Maksimalna brzina definira **amplitudu** B_i :
Maksimalna vrijednost $\sin^3 = 1$, kada je $\sin(wt) = 1$

$$\dot{q}_{max} = \frac{4B_i}{3} \longrightarrow B_i = \frac{3}{4} \dot{q}_{max}$$

Generiranje pobudnih trajektorija:

Naše istraživanje – definicija parametara

- Ograničenje akceleracije definira dopuštenu frekvenciju

$$\ddot{q}_i(t) = 4B_i w \sin^2(wt) \cos(wt)$$

Maksimalna vrijednost kada vrijedi: $\sin^2(wt) \cos(wt) = 1$



Nemoguće, za iste w i t vrijedi:

1. $\sin^2(wt) = 1; \cos(wt) = 0$

2. $\sin^2(wt) = 0; \cos(wt) = 1$



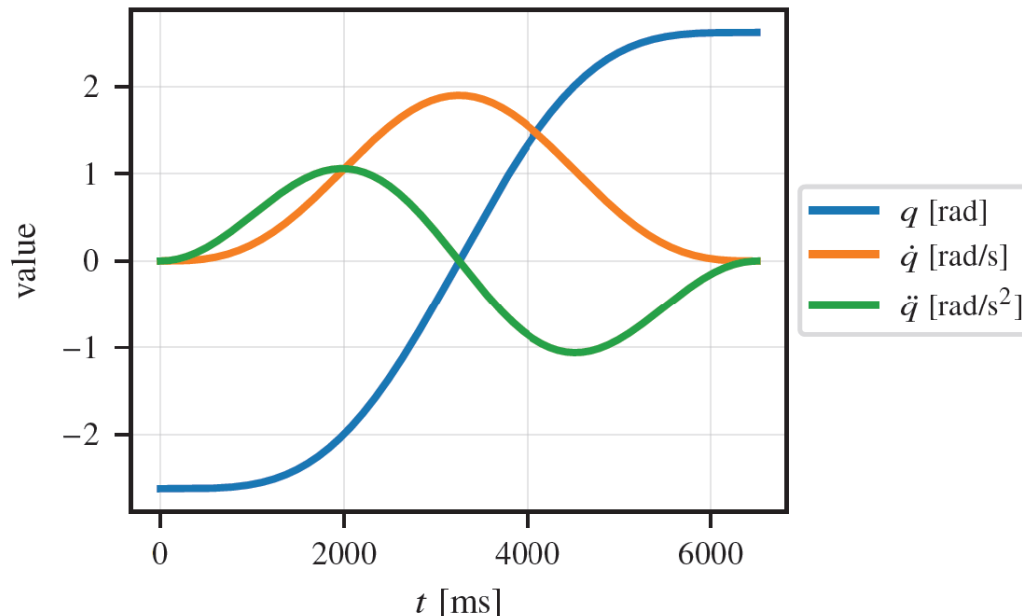
Uvijek vrijedi: $|\sin^2(wt) \cos(wt)| < 1$  Naš odabir je konzervativan:

$$\ddot{q}_{max} = 4B_i w \longrightarrow w = \frac{1}{4} \frac{\ddot{q}_{max}}{B_i} = \frac{1}{3} \frac{\ddot{q}_{max}}{\dot{q}_{max}}$$

Generiranje pobudnih trajektorija:

Naše istraživanje – definicija parametara

- Trajanje segmenta definirano je iz perioda pobude
- Interval: $t \in [0, T]$
- Prema definiciji perioda ($T = \frac{2\pi}{w}$): $T = 8\pi \frac{B_i}{\ddot{q}_{max}}$
- Trajektorija kreće i završava u stanju: $\dot{q} = \ddot{q} = 0$



Primjer:

$$q_{range} = [-2.618, 2.618]$$

$$\dot{q}_{limit} = 1.9 \frac{rad}{s}$$

$$\ddot{q}_{limit} = 1.5 \frac{rad}{s^2}$$

$$q_{start} = -2.618 \text{ rad}$$

Nakon generiranja trajektorija i pobude sustava...

- Skupimo podatke s robota ($\tau_{measured}$, position, velocity, acceleration ...)
- Formuliramo identifikacijski problem:

$$\hat{\theta} = \arg \min_{\theta} \sum_t \underbrace{\|\tau_{measured}(t) - \tau_{estimated}(t; \theta)\|^2}_{\text{IZMJERENI MOMENT}}$$

$\hat{\theta}$ → ESTIMIRANI PARAMETRIESTIMIRANI MOMENT

- Definiramo model koji identificiramo:

$$\tau_f = \begin{cases} \text{sign}(\dot{q})[F_c^+ + F_s^+ \cdot \Phi(\dot{q})] + F_v^+ \dot{q}, & \dot{q} > 0 \\ \text{sign}(\dot{q})[F_c^- + F_s^- \cdot \Phi(\dot{q})] + F_v^- \dot{q}, & \dot{q} < 0 \end{cases} \quad \Phi^\pm(\dot{q}) = \exp\left(-\left(\frac{|\dot{q}|}{v_s^\pm}\right)^{k_s^\pm}\right)$$

- Identificiramo model metodama optimizacije (npr. nelinearna regresija s pseudo-Huber gubitkom)

$$\hat{\theta} = \arg \min_{\theta} \sum_i \rho\left(\sqrt{w_i}(y_i - \hat{y}(\dot{q}_i; \theta))\right) \quad \rho(z) = \delta^2 \left(\sqrt{1 + \left(\frac{z}{\delta}\right)^2} - 1 \right)$$

Sinkronizacija više zglobova

Zašto svi zglobovi moraju završiti u istom trenutku?

- Jer želimo da TCP robota prati **jednu konzistentnu trajektoriju u prostoru**
- Kada bi u različita vremena zglobovi završavali, TCP putanja bila bi „lomljena”, gibanje bi izgledalo neprirodno, smjer kretanja mijenjao bi se usred gibanja
- Zahtjevi u implementaciji za glatku i predvidivu putanju je da svi zglobovi:
 1. Krenu kretanju zajedno
 2. Gibaju se koordinirano
 3. Završe kretanju zajedno
- Rješenje: svim zglobovima dodijelimo isto vrijeme t_f
- Ograničavajuća os: zglob koji treba najviše vremena određuje ukupno trajanje - $t_f = \max_i(t_{f,i})$

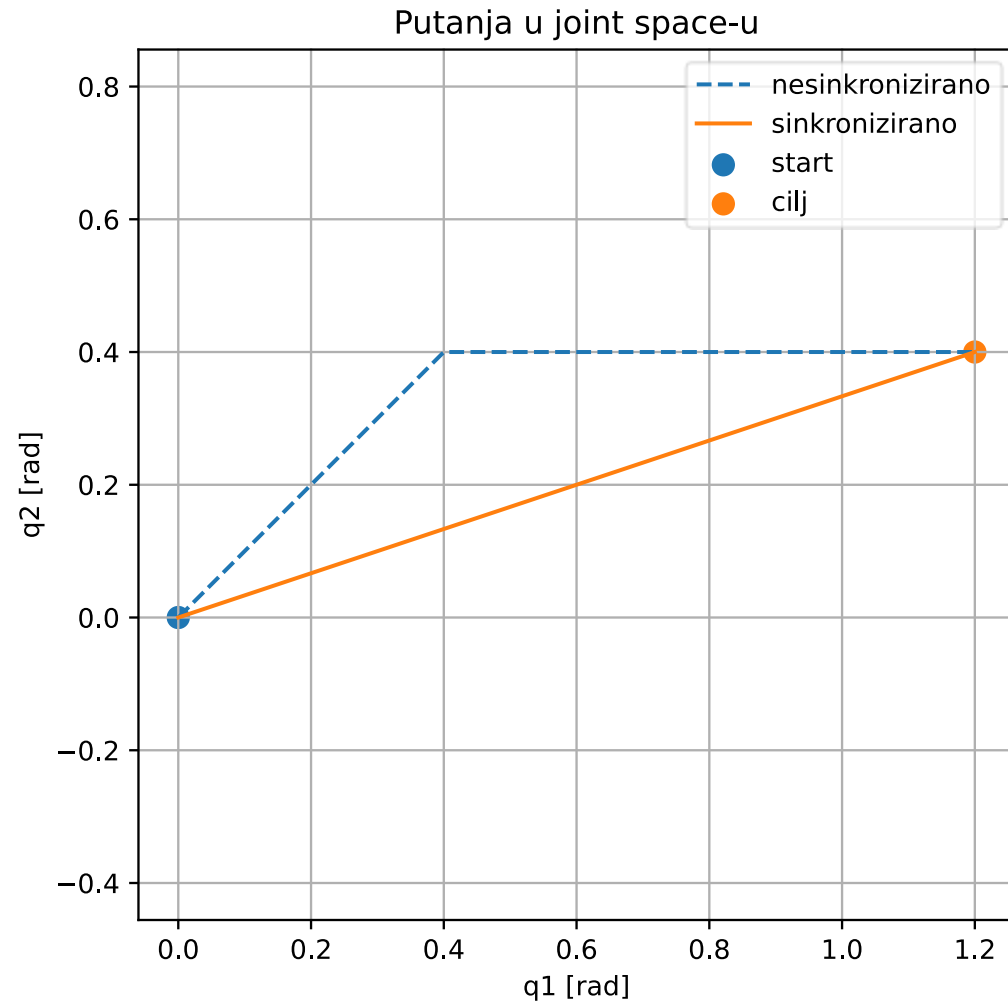
Sinkronizacija više zglobova

Nesinkronizirano gibanje (plavo):

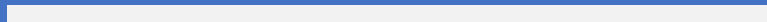
- zglobovi završavaju u različito vrijeme
- TCP ne prati jednu konzistentnu putanju

Sinkronizirano gibanje (narančasto):

- svi zglobovi koriste isto vrijeme
- putanja je **pravac između početne i završne konfiguracije**
- gibanje je koordinirano i predvidivo



Zadaci



Zadatak 1:

Generiranje point-to-point trajektorije u joint space-u – polinom prvog stupnja

- Za UR3e robota (6SSG) početne konfiguracije q_i i završne konfiguracije q_f potrebno je:
 1. Izračunati ukupni pomak svakog zgloba (D)
 2. Odrediti minimalno potrebno vrijeme trajektorije pojedinog zgloba tako da vrijedi uvjet $|\dot{q}(t)| \leq \dot{q}_{\max}$
 3. Generirati **sinkroniziranu trajektoriju** tako da svi zglobovi krenu i završe u istom trenu
 4. Za sve zglobove koristiti interpolaciju **polinomom prvog stupnja**
 5. Prikazati (plot) poziciju, brzinu i akceleraciju svih 6 zglobova
- Jednadžba trajektorije: $q(t) = q^i + s(t)(q^f - q^i)$
- Diskretizaciju trajektorije provesti na 1000 vremenskih koraka

$$q_i = [-1.571, -2.094, -1.571, 0.524, 1.571, 0.000] \text{ rad}$$

$$q_f = [-0.785, -2.094, 0.000, -1.571, 0.000, 0.000] \text{ rad}$$

Zadatak 1:

Generiranje point-to-point trajektorije u joint space-u – polinom prvog stupnja

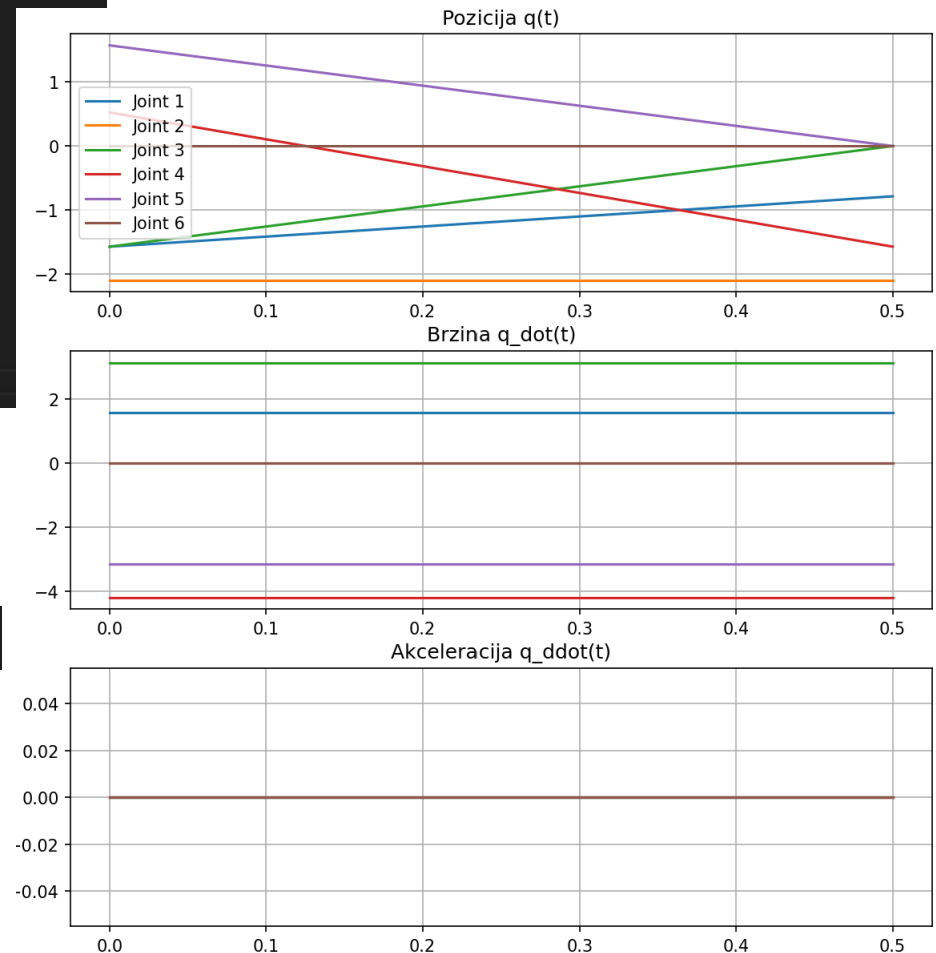
- Rješenje koje očekujemo:

```
trajectory_first_order.txt X
trajectory_first_order.txt > data
1  servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
2  servoj([-1.5702, -2.0940, -1.5694, 0.5219, 1.5694, 0.0000], t=0.0005)
3  servoj([-1.5694, -2.0940, -1.5679, 0.5198, 1.5679, 0.0000], t=0.0005)
4  servoj([-1.5686, -2.0940, -1.5663, 0.5177, 1.5663, 0.0000], t=0.0005)
5  servoj([-1.5679, -2.0940, -1.5647, 0.5156, 1.5647, 0.0000], t=0.0005)
6  servoj([-1.5671, -2.0940, -1.5631, 0.5135, 1.5631, 0.0000], t=0.0005)
7  servoj([-1.5663, -2.0940, -1.5616, 0.5114, 1.5616, 0.0000], t=0.0005)
8  servoj([-1.5655, -2.0940, -1.5600, 0.5093, 1.5600, 0.0000], t=0.0005)
9  servoj([-1.5647, -2.0940, -1.5584, 0.5072, 1.5584, 0.0000], t=0.0005)
10 servoj([-1.5639, -2.0940, -1.5568, 0.5051, 1.5568, 0.0000], t=0.0005)
11 servoj([-1.5631, -2.0940, -1.5553, 0.5030, 1.5553, 0.0000], t=0.0005)
12 servoj([-1.5623, -2.0940, -1.5537, 0.5009, 1.5537, 0.0000], t=0.0005)
13 servoj([-1.5616, -2.0940, -1.5521, 0.4988, 1.5521, 0.0000], t=0.0005)
14 servoj([-1.5608, -2.0940, -1.5506, 0.4967, 1.5506, 0.0000], t=0.0005)
15 servoj([-1.5600, -2.0940, -1.5490, 0.4946, 1.5490, 0.0000], t=0.0005)
```

Zapis pogodan za
slanje na UR robota

```
Minimalno vrijeme trajektorije: 0.500 s
Ograničavajuća os: Joint 3
Vrijeme po zglobovima: [0.25019157 0. 0.50006483 0.33342961 0.25003242 0. ]
```

Plot pozicije,
brzine i akceleracije



Zadatak 2:

Generiranje point-to-point trajektorije u joint space-u – polinom petog stupnja

- Za UR3e robota (6SSG) početne konfiguracije q_i i završne konfiguracije q_f potrebno je:
 1. Izračunati ukupni pomak svakog zgloba (D)
 2. Odrediti minimalno potrebno vrijeme trajektorije pojedinog zgloba tako da vrijedi uvjet $t_f = \max(t_{vel}, t_{acc})$
 3. Generirati **sinkroniziranu trajektoriju** tako da svi zglobovi krenu i završe u istom trenu
 4. Za sve zglobove koristiti interpolaciju **polinomom petog stupnja**
 5. Prikazati (plot) poziciju, brzinu i akceleraciju svih 6 zglobova
- Jednadžba trajektorije: $q(t) = q^i + s(t)(q^f - q^i)$
- Diskretizaciju trajektorije provesti na 1000 vremenskih koraka

$$q_i = [-1.571, -2.094, -1.571, 0.524, 1.571, 0.000] \text{ rad}$$

$$q_f = [-0.785, -2.094, 0.000, -1.571, 0.000, 0.000] \text{ rad}$$

Zadatak 2:

Generiranje point-to-point trajektorije u joint space-u – polinom petog stupnja

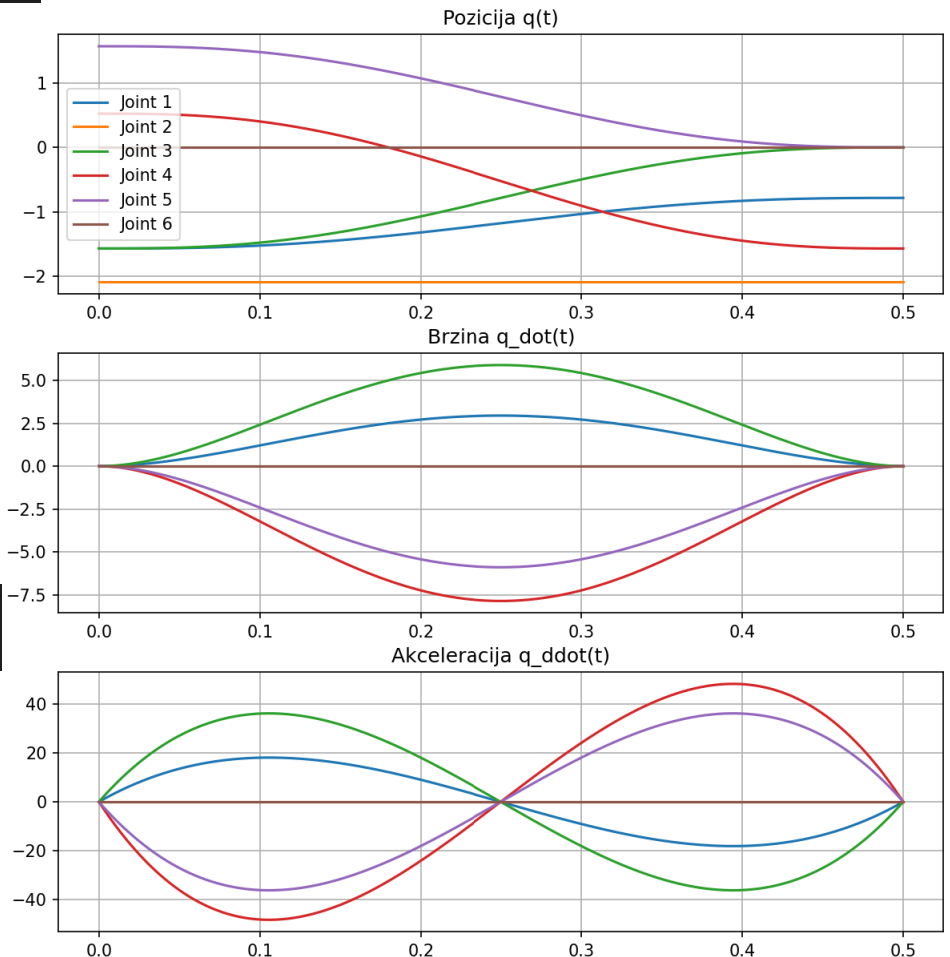
- Rješenje koje očekujemo:

```
trajectory_fifth_order.txt X
trajectory_fifth_order.txt > data
1  servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
2  servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
3  servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
4  servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
5  servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
6  servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
7  servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
8  servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
9  servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
10 servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
11 servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
12 servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
13 servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
14 servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
15 servoj([-1.5710, -2.0940, -1.5710, 0.5240, 1.5710, 0.0000], t=0.0005)
```

Zapis pogodan za
slanje na UR robota

```
Minimalno vrijeme: 1.201 s
Ograničavajuća os: Joint 3
Vrijeme po zglobovima: [0.84958971 0. 1.20111908 0.98078765 0.84931944 0. ]
Kritično ograničenje: akceleracija
```

Plot pozicije,
brzine i akceleracije



Zadatak 3:

Slanje trajektorija na robota (simulacija):

- Potrebno je implementirati Python skriptu koja će ostvariti komunikaciju s **UR robotom putem TCP/IP-a** te izvršiti unaprijed generiranu trajektoriju u zglobnom prostoru (robot simuliran – PolyScope)
- Zadatak uključuje sljedeće korake
 1. učitati trajektoriju iz .txt datoteke (servoj naredbe u radijanima)
 2. uspostaviti socket vezu s robotom (real-time port)
 3. poslati trajektoriju kao URScript program
 4. omogućiti da robot izvršava trajektoriju vlastitom kontrolnom frekvencijom
- Potrebno je pustiti datoteke generirane u **prvom** i **drugom** zadatku i usporediti ponašanje robota
- **Napomena:** kod implementacije koda na **pravom robotu** paziti da je početna konfiguracija namještena u početku slanja programa



Zadatak 4:

Generiranje point-to-point trajektorije u task space-u – polinom petog stupnja

- Za UR3e robota (6SSG) potrebno je generirati **point-to-point trajektoriju TCP-a u task space-u** koristeći polinom 5. reda te uzeti u obzir ograničenja brzine i akceleracije robota
- Zadano je:
 - početna i konačna pozicija TCP-a p_i, p_f te početna i konačna orijentacija R_i, R_f
 - ograničenja:
 - maksimalna translacijska brzina $v_{max} = 0.5 \text{ m/s}$
 - maksimalna akceleracija $a_{max} = 1.0 \text{ m/s}^2$
 - maksimalna kutna brzina $\omega_{max} = 1.0 \text{ rad/s}$
- Interpolacija translacije: $p(t) = p_i + s(t)(p_f - p_i)$, interpolacija rotacije: SLERP (scipy.spatial.transform)

$$p_i = [-0.30, -0.120, 0.3] \text{ m}, p_f = [-0.225, -0.120, 0.33] \text{ m}$$

$$R_i = [90, 0, 0]^\circ, R_f = [120, 0, 45]^\circ$$

Zadatak 4:

Generiranje point-to-point trajektorije u task space-u – polinom petog stupnja

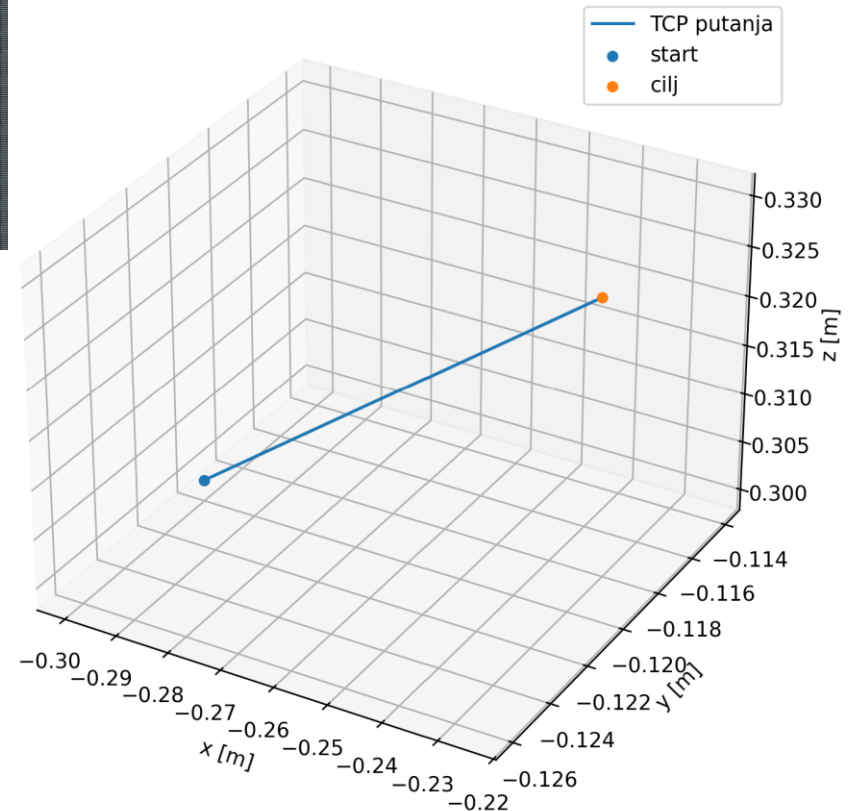
- Rješenje koje očekujemo:

```
trajectory_taskspace.txt X
trajectory_taskspace.txt > data
1 servoj(get_inverse_kin(p[-0.3000, -0.1200, 0.3000, 1.5708, 0.0000, 0.0000]), t=0.0080, 100
2 servoj(get_inverse_kin(p[-0.3000, -0.1200, 0.3000, 1.5708, 0.0000, 0.0000]), t=0.0080, 100
3 servoj(get_inverse_kin(p[-0.3000, -0.1200, 0.3000, 1.5708, 0.0000, 0.0000]), t=0.0080, 100
4 servoj(get_inverse_kin(p[-0.3000, -0.1200, 0.3000, 1.5708, 0.0000, 0.0000]), t=0.0080, 100
5 servoj(get_inverse_kin(p[-0.3000, -0.1200, 0.3000, 1.5708, 0.0000, 0.0000]), t=0.0080, 100
6 servoj(get_inverse_kin(p[-0.3000, -0.1200, 0.3000, 1.5709, 0.0001, 0.0001]), t=0.0080, 100
7 servoj(get_inverse_kin(p[-0.3000, -0.1200, 0.3000, 1.5709, 0.0001, 0.0001]), t=0.0080, 100
8 servoj(get_inverse_kin(p[-0.3000, -0.1200, 0.3000, 1.5709, 0.0002, 0.0001]), t=0.0080, 100
9 servoj(get_inverse_kin(p[-0.3000, -0.1200, 0.3000, 1.5710, 0.0004, 0.0002]), t=0.0080, 100
10 servoj(get_inverse_kin(p[-0.3000, -0.1200, 0.3000, 1.5711, 0.0005, 0.0003]), t=0.0080, 100
11 servoj(get_inverse_kin(p[-0.2999, -0.1200, 0.3000, 1.5712, 0.0007, 0.0004]), t=0.0080, 100
12 servoj(get_inverse_kin(p[-0.2999, -0.1200, 0.3000, 1.5714, 0.0009, 0.0005]), t=0.0080, 100
13 servoj(get_inverse_kin(p[-0.2999, -0.1200, 0.3000, 1.5715, 0.0011, 0.0007]), t=0.0080, 100
```

Zapis pogodan za
slanje na UR robota

```
D = 0.081 m
alpha = 0.936 rad
t_vel = 0.303 s
t_acc = 0.683 s
t_rot = 1.756 s
t_f = 1.756 s
```

Task-space putanja (quintic)



Plot putanje u task space-u

Zadatak 5:

Slanje trajektorija na robota (simulacija):

- Potrebno je implementirati Python skriptu koja će ostvariti komunikaciju s **UR robotom putem TCP/IP-a** te izvršiti unaprijed generiranu trajektoriju u zglobnom prostoru (robot simuliran – PolyScope)
- Zadatak uključuje sljedeće korake
 1. učitati trajektoriju iz .txt datoteke (servoj naredbe u radijanima)
 2. uspostaviti socket vezu s robotom (real-time port)
 3. poslati trajektoriju kao URScript program
 4. omogućiti da robot izvršava trajektoriju vlastitom kontrolnom frekvencijom
- Potrebno je pustiti datoteku generiranu u **četvrtom** zadatku i usporediti ponašanje robota
- **Napomena:** kod implementacije koda na **pravom robotu** paziti da je početna konfiguracija namještena u početku slanja programa

